

TamaGo

Bare metal Go

Secure unikernels:
From Embedded Systems to Confidential VMs

Andrea Barisani

<https://andrea.bio>

andrea@inversepath.com

andrea.barisani@reversesec.com

Information Security Engineer and Researcher

Co-Founder: **INVERSE**  **PATH** (2005 - 2017)

Head of Hardware Security:  **WI**

Head of Security Engineering: 

Co-creator, USB armory:



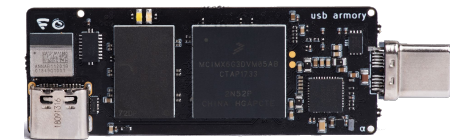
Speaker at too many conferences...

Background focused on security auditing and engineering on safety critical systems in the automotive, avionics, industrial domains.

Now also hacking compilers and runtimes...



Motivation: USB armory firmware



The USB armory is a tiny, but powerful, embedded platform for personal security applications.



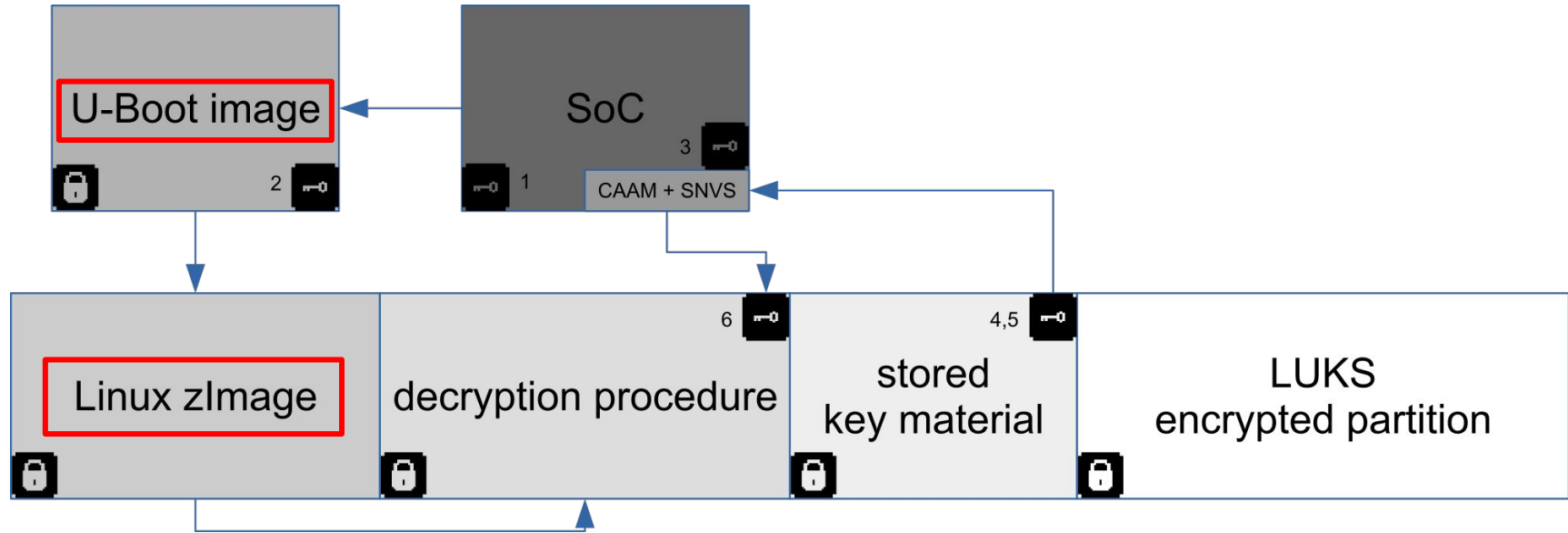
Designed to fit in a pocket, laptops, PCs, server and networks (LAN model with PoE).

The USB armory targets the following primary applications:

- Encrypted storage solutions
- Hardware Security Module (HSM)
- Enhanced smart cards
- Electronic vaults (e.g. cryptocurrency wallets) and key escrow services
- Authentication, provisioning, licensing tokens
- USB firewall



Reducing the attack surface



Typical secure booted firmware with authentication and confidentiality on an NXP i.M6UL.



Killing C

When security matters software and hardware optimizations matter less.

This means that less constrained hardware (e.g. SoCs in favor of MCUs) and higher level code are perfectly acceptable.

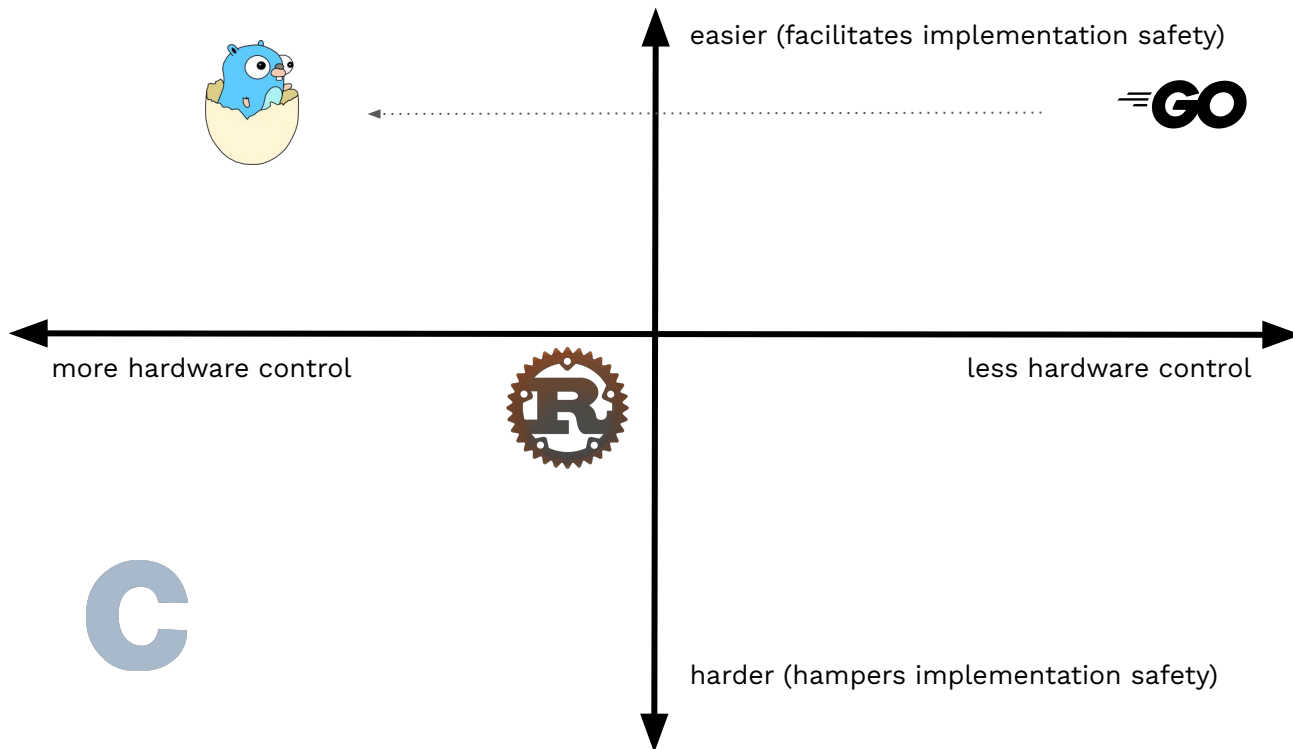
However high level programming typically entails several layers (e.g. OS, libraries) to serve runtime execution.

TamaGo spawns from the desire of **reducing the attack surface** of embedded systems firmware by **removing any runtime dependency on C code and inherently complex Operating Systems**.

In other words we want to **avoid shifting complexity around** and run a **higher level language**, such as Go in our effort, **directly on the bare metal**.



Speed vs Safety



Disclaimer: chart presented for discussion and not to claim that language X is better than language Y, also scale is subjective.



Unikernels / library OS

Unikernels¹ are a single address space image to executed a “library operating system”, typically running under bare metal.

The focus is reducing the attack surface, carrying only strictly necessary code.

“True” unikernels are mostly unicorns, as a good chunk of available ones do not fit in this category and represent “fat” unikernels running under hypervisors and/or other (mini) OSes And just shift around complexity (e.g. the app is PID 1).

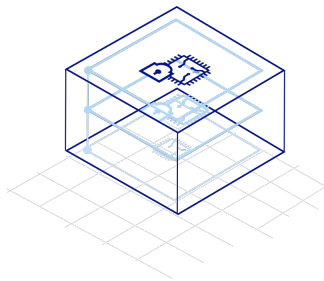
Apart for some exceptions there is always still a lot of C/dependencies involved in the underlying OS, drivers or hypervisor.

Running or importing *BSD kernels

Rump kernels (NetBSD based)
OSv (re-uses code from FreeBSD)

Running under hypervisor and 3rd party kernel

MirageOS (Solo5)
ClickOS (MiniOS)



Running under hypervisor

Nanos (Xen/KVM/Qemu) HalVM (Haskell, Xen)
LING (Erlang, Xen) RustyHermit (KVM)

Bare metal

GRISP (Erlang)
IncludeOS

¹ <https://en.wikipedia.org/wiki/Unikernel>



TamaGo in a nutshell

- A **minimally**¹ patched Go distribution to enable `GOOS=tamago` support, which provides freestanding execution on `GOARCH={amd64|arm|arm64|loong64|riscv64}` bare metal.
- A set of external packages² for architecture, board/SoC support, including hardware initialization (e.g. boot).

<code>cloud_hypervisor/vm</code>	Cloud Hypervisor
<code>firecracker/microvm</code>	Firecracker
<code>google/gcp</code>	Google Compute Engine
<code>qemu/microvm</code>	QEMU microvm
<code>uefi/x64</code>	UEFI

AMD64

<code>nuvoton/nuc980ioot</code>	NuMaker-IIoT-NUC980G2
<code>nxp/mx6ullevk</code>	NXP MCIMX6ULL-EVK
<code>raspberrypi/pi1</code>	Raspberry Pi 1
<code>raspberrypi/pi2</code>	Raspberry Pi 2
<code>raspberrypi/pizero</code>	Raspberry Pi Zero
<code>usbarmory/mk2</code>	USB armory Mk II (USB+LAN)

ARM

<code>microchip/lan969x</code>	Microchip LAN969x
<code>nxp/imx8mpevk</code>	NXP 8MPLUSLPD4-EVK

ARM64

<code>aifoundry/erbium_emu</code>	AI Foundry Erbium	(GOSOFT=1)
<code>aifoundry/sys_emu</code>	AI Foundry ET-SoC-1	(GOSOFT=1)
<code>qemu/sifive_u</code>	QEMU SiFive HiFive Unleashed	

RISCV64

WiP (compiler support only)

LOONG64



¹ <https://github.com/usbarmory/tamago-go>

² <https://github.com/usbarmory/tamago>



Releases

v1.26.4

Latest

 abarisan released this last week  v1.26.4   5e5171c

This release features:

- improved CPU idle management for `amd64`, aimed at reducing CPU usage on multi-core (e.g. SMP) unikernels
- improved AMD SEV-SNP support for vCPU creation
- improved amd64 SMP support for external AP initialization (e.g. UEFI, AMD SEV-SNP)

Major changes for [tamago package API](#):

- `amd64`: `(*CPU).Wake` new function to wake up idle CPUs ([6a9fc7e](#))
- `amd64`: `(*CPU).GOMAXPROCS` new function for runtime system thread configuration ([855a632](#), [6e32384](#))
- `goos`: `runtime/goos.Wake` new optional function to wake up idle CPUs ([6a9fc7e](#))

Major changes for [tamago package API](#) to add AMD SEV-SNP support under SMP:

- `kvm/sev`: `(*VMSA).Init` new function for VMSA initialization ([7ab1167](#), [f4b3053](#))
- `kvm/sev`: `(*GHCB).HypervisorFeatures` new function to get hypervisor feature support ([229a094](#))
- `kvm/sev`: `(*GHCB).CreateAP` new function for AP creation ([7ab1167](#))
- `kvm/sev`: `(*GHCB).RemoveAP` new function for AP removal ([7ab1167](#))
- `kvm/sev`: `(*GHCB).VMSA` new field for shared guest/psp VMSA buffers DMA region ([7ab1167](#))
- `kvm/sev`: `(*GHCB).CPU` new field for vCPU GPA access ([8b31c87](#))
- `kvm/sev`: `(*GHCB).Init` function removed in favor of opportunistic vCPU-bound initialization ([8b31c87](#))
- `kvm/sev`: `SEVStatus.Features` new field to get SEV Guest feature selection ([229a094](#))

Major changes for tamago package internals:

- `amd64`: handle user defined interrupts regardless of ISR presence ([fb91823](#))
- `amd64`: segment registers no longer reloaded on IDT setup ([04fb603](#))

Full Changelog: [v1.26.3...v1.26.4](#)

This release requires `GOOS=tamago` support in the Go distribution with [tamago-go1.26.4](#) or later releases, it has been tested with [tamago-go1.26.4](#).

tamago-go1.26.4

Latest

This is a release for the Go distribution as modified by the [TamaGo framework](#), which enables compilation and execution of unencumbered Go applications on bare metal processors (AMD64, ARM, ARM64, RISCv64).

This release adds `GOOS=tamago` support to [Go 1.26.4](#).

Major changes for `GOOS=tamago` [runtime API](#):

- `goos.Wake`: new function to wake processors when halted by `goos.Idle` ([20585ee](#))
- `goos.Task`: prevent panic on excessive load by never exceeding 1:1 M/P ratio ([ad17f41](#), [0055003](#), [b17a2ce](#))
- `runtime.LockOSThread`: function disabled ([f88b67c](#))

Major changes for `GOOS=tamago` runtime internals:

Full Changelog: [tamago-go1.26.3...tamago-go1.26.4](#)

Upstream comparison: [go1.26.4...tamago-go1.26.4](#)

Release binaries

The release assets include the pre-compiled modified Go distribution required to compile [applications using the TamaGo framework](#) on amd64 and armv7l (such as the [USB armory Mk II](#)) Linux hosts.

Installation:

```
sudo tar -xvf tamago-go<version>.linux-<arch>.tar.gz -C /
```

You should now be able to compile the [example application](#) by setting the `TAMAGO` environment variable as follows:

```
export TAMAGO=/usr/local/tamago-go/bin/go
```

▼ Assets 4

 tamago-go1.26.4.linux-amd64.tar.gz	sha256:829a4ea1eaf72a... 	64.6 MB	last week
 tamago-go1.26.4.linux-armv7l.tar.gz	sha256:1609f6848dae4c... 	62.8 MB	last week
 Source code (zip)			last week
 Source code (tar.gz)			last week



Similar efforts

Past/existing efforts

(all these projects greatly supported us in proving feasibility and identify TamaGo unique approach, diversity is good)

Biscuit (inactive) - <https://github.com/mit-pdos/biscuit>

Go kernel for non-Go software underneath, larger scope, needs two C bootloaders, hijacks `GOOS=linux`, only for `GOARCH=amd64`, redoes memory allocation and threading.

G.E.R.T (inactive) - <https://github.com/ycoroneos/G.E.R.T>

ARM adaptation of Biscuit but without non-Go software support, needs two C bootloaders, hijacks `GOOS=linux` for `GOARCH=arm`, redoes memory allocation and threading.

AtmanOS (inactive) - <https://github.com/atmanos>

Similar to TamaGo but targets the Xen hypervisor, adds `GOOS=atman` but with limited runtime support.

TinyGo (active) - <https://github.com/tinygo-org>

LLVM based compiler (not original one) aimed at MCUs and minimal footprint, does not support the entire runtime and Go language support differs from standard Go.

Embedded Go (active) - <https://github.com/embeddedgo>

Similar to TamaGo but targets ARMv7-M/ARMv8-M (w/ Thumb2) adding new support for it, as not native to Go. Adds `GOOS=noos` `GOARCH=thumb`.

Egg OS (inactive) - <https://github.com/icexin/eggos>

Targets x86, uses native compiler and wraps `GOOS=linux` syscalls back to Go.



Enabling trust

TamaGo not only proves that it is possible to have a bare metal Go runtime, but does so with **clean and minimal modifications against the original Go distribution**².

Much of the effort has been placed to understand whether Go bare metal support can be achieved without complex re-implementation of memory allocation, threading, ASM/C OS primitives that would “pollute” the Go runtime to unacceptable levels.

Less is more. Complexity is the enemy of verifiability.

The acceptance of this (and similar) efforts hinges on maintainability, ease of review, clarity, simplicity and **trust**.

- ★ Designed to achieve upstream inclusion and with commitment to always sync to latest Go release.
- ★ ~8000 LOC of changes against Go distribution with clean separation from other GOOS support.
- ★ Strong emphasis on code reuse from existing architectures of standard Go runtime, see [Internals](#)¹.
- ★ Requires only one import (“library OS”) on the target Go application.
- ★ Supports unencumbered Go applications with native runtime and stdlib support.
- ★ In addition to the compiler, aims to provide a complete set of peripheral drivers for SoCs.

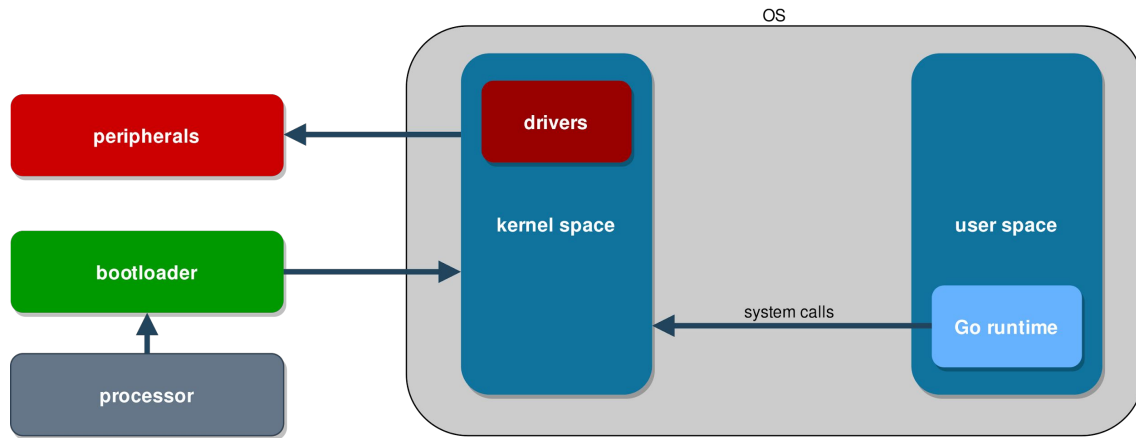
¹ <https://github.com/usbmarmory/tamago/wiki/Internals>

² Which by the way is self-hosted and has reproducible builds.

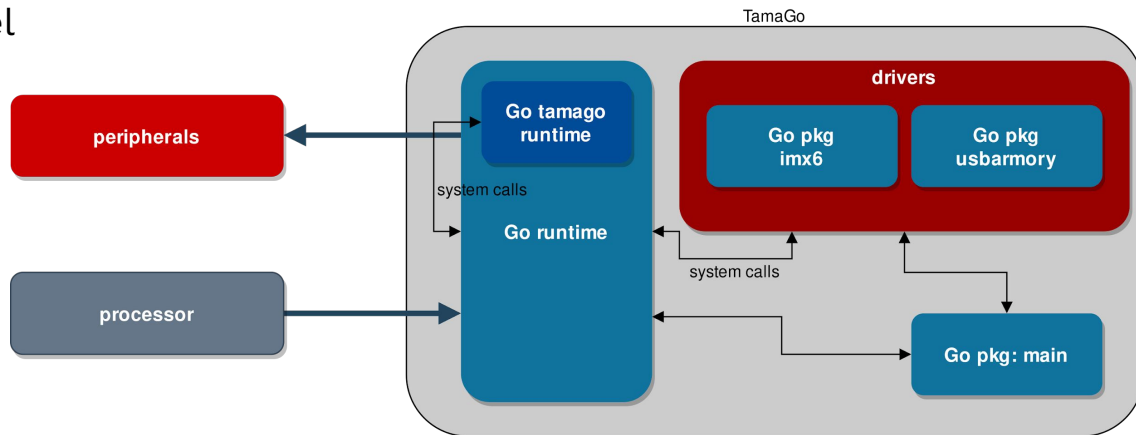


TamaGo

Traditional OS



TamaGo unikernel



runtime/goos API

Thanks to the upstreaming proposal, the implementation of `GOOS=tamago` has been refined in a “Rosetta Stone” for freestanding Go.

The Go runtime modifications are board and architecture independent, no language or standard library limitations.

MMU initialization, exception and IRQ handling are implemented outside the Go runtime and imported as architecture packages (e.g. `amd64`, `arm`). Low-level APIs (e.g. `TrustZone`), networking, can be regular packages.

This entails that hardware ports can be achieved through external packages, whether bare metal, OS kernel space, OS user space, TEE monitor or TEE applet.

The API is defined as `runtime/goos` overlay:

`GOOSPKG=github.com/usbarmory/tamago`

<https://pkg.go.dev/github.com/usbarmory/tamago/goos>

Go runtime “Rosetta Stone”

Proposal Details

I propose the addition of a new `GOOS` target, such as `GOOS=none`, to allow Go runtime execution under specific application defined exit functions, rather than arbitrary OS syscalls, enabling freestanding execution without direct OS support.

This is currently implemented in the [GOOS=tamago](#) project, but for reasons laid out in the *Proposal Background* section it is proposed for upstream inclusion.

Go applications built with `GOOS=none` would run on bare metal, without any underlying OS. All required support is provided by the Go runtime and external driver packages, also written in Go.

Go runtime changes

Note

The changes are also documented in package [tamago/doc](#).

A working example of all proposed changes can be found in the [GOOS=tamago implementation](#).

Board support packages or applications would be required (only under `GOOS=none`) to define the following functions to support the runtime.

If the use of `go:linkname` is undesirable different strategies are possible, right now `linkname` is used as convenient way to have externally defined functions being directly invoked in the runtime early on.

These hooks act as a “Rosetta Stone” for integration of a freestanding Go runtime within an arbitrary environment, whether bare metal or OS supported.

For bare metal examples see the following packages: [usbarmory](#), [uefi](#), [microvm](#).

For OS supported examples see the following tamago packages: [linux](#), [applet](#).

Index

GO reference

- [Variables](#)
- [func GetRandomData\(b \[\]byte\)](#)
- [func Hwinit0\(\)](#)
- [func Hwinit1\(\)](#)
- [func InitRNG\(\)](#)
- [func Nanotime\(\) int64](#)
- [func Printk\(c byte\)](#)

<https://github.com/golang/go/issues/73608>



Go runtime/goos API

func CPUInit()

Handles immediate startup CPU initialization.

```
TEXT ·Init(SB),NOSPLIT|NOFRAME,$0          riscv64/init.s
    // disable interrupts
    MOV     $0, T0
    CSRRW   T0, SIE, ZERO
    CSRRW   T0, MIE, ZERO
    MOV     $0x7fff, T0
    CSRRC   T0, MSTATUS, ZERO

    // enable FPU
    MOV     $(1<<13), T0
    CSRRS   T0, MSTATUS, ZERO

    // set stack pointer
    MOV     runtime/goos·RamStart(SB), X2
    MOV     runtime/goos·RamSize(SB), T1
    MOV     runtime/goos·RamStackOffset(SB), T2
    ADD     T1, X2
    SUB     T2, X2

    JMP     _rt0_tamago_start(SB)
```

func GetRandomData(b []byte)

Generates len(b) random bytes and writes them into b.

func InitRNG()

Initializes random number generation.

```
// defined in rng.s          amd64/rng.go
func rdrand() uint32

// GetRandomData returns len(b) random bytes gathered
// from the RDRAND instruction.
func GetRandomData(b []byte) {
    read := 0
    need := len(b)

    for read < need {
        read = rng.Fill(b, read, rdrand())
    }
}

func InitRNG() {}
```



Go runtime/goos API

func Hwinit0()

Takes care of lower level initialization triggered before runtime setup (pre World start).

func Hwinit1()

Takes care of lower level initialization triggered early in runtime setup (post World start).

```
arm/init.go
func Hwinit0() {
    if int(read_cpsr()) & 0x1f != SYS_MODE {
        // initialization required only when in PL1
        return
    }
    vfp_enable()
}
```

Hwinit0 should be defined using Go's Assembler to retain Go's commitment to backward compatibility.

Otherwise care must be taken as the lack of World start does not allow memory allocation.

```
firecracker/microvm.go
func Hwinit1() {
    // initialize CPU
    AMD64.Init()

    // initialize I/O APICs
    IOAPIC0.Init()

    // initialize serial console
    UART0.Init()

    runtime.Exit = func(_ int32) {
        AMD64.Reset()
    }
}
```



Go runtime/goos API

func Nanotime() int64

Returns the system time in nanoseconds.

func Printk(c byte)

Handles character printing to standard output.

```
func Nanotime() int64 {
    return ARM.GetTime()
}
```

imx6ul/timer.go

```
func (cpu *CPU) GetTime() int64 {
    return int64(float64(read_cntpct())*cpu.TimerMultiplier) + cpu.TimerOffset
}
```

arm/timer.go

```
func Printk() {
    imx6ul.UART2.Tx(c)
}
```

usbarmory/mk2/console.go

Must be defined using Go's Assembler to retain Go's commitment to backward compatibility.

Otherwise care must be taken as the lack of World start does not allow memory allocation.



Go runtime/goos API

RAM layout defined by architecture and board packages

```
var RamStart uint
var RamSize uint
var RamStackOffset uint
```

[imx6ul/timer.go](#)

```
var RamStart uint32 = MMDC_BASE
```

[usbarmory/mk2/mem.go](#)

```
var RamSize uint32 = 512 << 20
```

CPU idle time management (optional)

```
var Idle func(until int64)
```

[arm/arm.go](#)

```
func (cpu *CPU) Init(vbar uint32) {
    goos.Idle = func(pollUntil int64) {
        // we have nothing to do forever
        if pollUntil == math.MaxInt64 {
            cpu.WaitInterrupt()
        }
    }
}
```

Runtime termination (optional)

```
var Exit func(code int32)
```

Network socket implementation (optional)

```
var SocketFunc func(ctx context.Context,
    net string,
    family, sotype int,
    laddr, raddr Addr) (interface{}, error)
```

[go-net/gvisor.go](#)

```
func (iface *Interface) Socket(ctx context.Context ...) {
    ...
    gonet.DialContextTCP(ctx, stack, rFullAddr, proto)
}
```

[tamago-example/network/imx-enet.go](#)

```
// hook interface to Go runtime
net.SocketFunc = iface.Socket
```



HW/OS threading (optional)

```
var Task func(sp, mp, gp, fn unsafe.Pointer)
var Wake func(procid uint64)
```

Initialization of supplemental cores (APs) is outside the Go distribution, only `Task` is required as `runtime/goos` hook and is invoked with `GOMAXPROCS > 1`.

`Wake` allows mode advanced `Idle`.

```
> info
Runtime .....: go1.26.4 tamago/amd64 GOMAXPROCS=8
RAM .....: 0x10000000-0x50000000 (1024 MiB)
Board .....: microvm
CPU .....: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz
Cores .....: 8
Frequency ....: 2.8032 GHz
VirtIO Net0 ...: 52:54:00:12:34:56
```

```
> smp 100
8 cores detected, launching 100 goroutines from CPU 0
CPU 1 8: ██████████
CPU 3 14: ██████████
CPU 5 9: ██████████
CPU 6 13: ██████████
CPU 7 24: ██████████
CPU 0 13: ██████████
CPU 2 19: ██████████
Total 100 (307.813µs)
```

`amd64/smp.go`

```
// Task schedules a goroutine on a previously initialized
// Application Processor.
func (cpu *CPU) Task(sp, _, gp, fn unsafe.Pointer) {
    t := &task{
        sp: uint64(uintptr(sp)),
        gp: uint64(uintptr(gp)),
        pc: uint64(uintptr(fn)),
    }

    ...

    t.Write(taskAddress)

    // set last initialized CPU, signal task via NMI
    cpu.init += 1
    cpu.LAPIC.IPI(cpu.init, 0, lapic.ICR_DLV_NMI)
}

// ID returns the processor identifier.
func (cpu *CPU) ID() uint64 {
    return uint64(cpu.LAPIC.ID())
}

// Wake issues a wake interrupt to the target processor.
func (cpu *CPU) Wake(procid uint64) {
    cpu.LAPIC.IPI(int(procid), IRQ_WAKEUP, lapic.ICR_DLV_IRQ)
}
```



Re-implementing Linux as “2nd class port”

CPUInit()	→ sys_mmap(ramStart, ramSize, PROT_READ PROT_WRITE, ...)
GetRandomData(b []byte)	→ sys_getrandom(b, len(b))
Nanotime() int64	→ sys_clock_gettime()
Printk(c byte)	→ sys_write(c)
Task(sp, mp, gp, fn unsafe.Pointer)	→ sys_clone(flags, sp, mp, gp, fn)
Exit(code int32)	→ sys_exit_group(code)

runtime/goos/linux_user_arm64.s

```
TEXT CPUInit(SB),NOSPLIT|NOFRAME,$0
    MOVQ    ·RamStart(SB), DI
    MOVQ    ·RamSize(SB), SI
    MOVL    $0x3, DX          // PROT_READ  | PROT_WRITE
    MOVL    $0x22, R10        // MAP_PRIVATE | MAP_ANONYMOUS
    MOVL    $0xffffffff, R4
    MOVL    $0, R9
    MOVL    $SYS_mmap, AX
    SYSCALL
    ...
    JMP     runtime·rt0_tamago_start(SB)

// func sys_exit_group(code int32)
TEXT ·sys_exit_group(SB), $0-4
    MOVW    code+0(FP), DI
    MOVD    $SYS_exit_group, AX
    SYSCALL
    RET
```

A single unified Go file allows hookup of TamaGo runtime API back to Linux.

Architecture specific Go assembly files, which are verbatim copies of relevant functions from `sys_linux_{amd64,arm,arm64,loong64,riscv64}.s`, provide Linux system call hooks.

This approach allow creation of support for GOOS=tamago in OS userspace for testing and user/linux packages.



tamago-go/src/runtime/goos/linux_user.go

```
const (
    bits = 32 << (^uint(0) >> 63) / 8

    ArenaBaseOffset      = 0
    HeapAddrBits         = (8-bits)*3 + bits*5
    LogHeapArenaBytes    = (2 + 20)
    LogPallocChunkPages = 9
    MinPhysPageSize     = 4096
    StackSystem          = 0
)

var (
    RamStart      uint = 0x80000000
    RamSize       uint = 512 << 20
    RamStackOffset uint = 0x100

    Bloc      = uintptr(RamStart)
    Exit      = sys_exit_group
    Idle      func() int64
    ProcID    func() uint64
    Wake      func(uint64)

    Hwinit0 = func() {}
    InitRNG  = func() {}
    Nanotime = sys_clock_gettime
    Hwinit1  = func() {}
)

// defined in linux_user*.s
func CPUInit()
func sys_exit_group(code int32)
func sys_write(c *byte)
func sys_clock_gettime() (ns int64)
func sys_getrandom(b []byte, n int)

//go:noescape
func clone(flags int32, stk, mp, gp, fn unsafe.Pointer) int32

func GetRandomData(b []byte) {
    sys_getrandom(b, len(b))
}

// preallocated memory to avoid malloc during panic
func a [1]byte

func Printk(c byte) {
    a[0] = c
    sys_write(&a[0])
}

func Task = func(sp, mp, gp, fn unsafe.Pointer) {
    clone(cloneFlags, sp, mp, gp, fn)
}
```

This is the entire Go definition for the runtime/goos overlay which implements Linux for GOOS=tamago GOARCH=.

Userspace execution

```
package main

import (
    "fmt"
    "net"
    "os"

    _ "github.com/usbarmory/tamago/user/linux"
)

func main() {
    _, err := net.Dial("tcp", "8.8.8.8:53")
    fmt.Printf("** I can't get out!    ;-( ** %s\n", err)

    _, err = os.ReadFile("/etc/passwd")
    fmt.Printf("** I can't get out!    ;-( ** %s\n", err)
}
```

```
$ GOOS=tamago ./bin/go run main.go
```

```
** I can't get out!    ;-( ** dial tcp 8.8.8.8:53: net.SocketFunc is nil
** I don't see the FS! ;-( ** open /etc/passwd: No such file or directory
```

Any imported package can implement the required runtime changes with OS supervision instead of bare metal drivers.

This allows user space execution with the benefit of system calls isolation.

The executable cannot leverage on the Go runtime to directly access OS resources.

This allows custom APIs for networking, RNG, time handling and is also used for Trusted Applet implementation (see later GoTEE coverage).

<https://github.com/usbarmory/tamago/wiki#userspace-targets>

Go distribution testing for GOOS=tamago

```
$ cd tamago-go
$ GO_BUILDER_NAME=tamago GOOS=tamago GOARCH=arm64 ./bin/go tool dist test
```

```
##### Testing packages.
ok      archive/tar      1.465s
ok      archive/zip      156.222s
ok      bufio            0.683s
ok      bytes            88.814s
ok      cmp              0.106s
ok      compress/bzip2   0.299s
ok      compress/lzw     0.501s
ok      compress/zlib    3.909s
ok      container/heap   0.116s
ok      container/list   0.103s
ok      container/ring   0.099s
ok      context          0.195s
ok      crypto           0.105s
ok      crypto/aes       0.175s
ok      crypto/cipher    0.124s
ok      crypto/des       0.149s
ok      crypto/dsa       814.124s
ok      crypto/ecdh      3.932s
ok      crypto/ecdsa     79.698s
ok      crypto/ed25519   4.154s
ok      crypto/elliptic  14.773s
ok      crypto/hmac      0.218s
...
ok      cmd/internal/src 0.138s
ok      cmd/internal/test2json 0.115s
ok      cmd/link          0.216s
ok      cmd/link/internal/benchmark 0.161s
ok      cmd/link/internal/ld 0.346s
ok      cmd/link/internal/loader 0.268s
ok      cmd/nm           0.254s
ok      cmd/objdump      0.249s
ok      cmd/pack         0.224s
ok      cmd/pprof        0.321s
ok      cmd/relnote      0.164s
ok      cmd/trace        0.257s
ok      cmd/vet          0.290s
```

Demonstrates importing, execution and test compliance for the entire Go stdlib on `GOARCH={amd64,arm,arm64,loong64,riscv64}`.

Runs under Linux (natively or using qemu via `binfmt_misc`) bridging required external functions with Linux syscalls.

In-memory filesystem and networking support `testdata` access and network tests.

<https://github.com/usbarmory/tamago/wiki/Compatibility>

Go low level access

```
func (hw *BEE) Init() {
    hw.mu.Lock()
    defer hw.mu.Unlock()

    hw.ctrl = hw.Base + BEE_CTRL
    hw.key = hw.Base + BEE_AES_KEY0_W0

    reg.Set(hw.ctrl, CTRL_CLK_EN)      // enable clock
    reg.Set(hw.ctrl, CTRL_SFTRST_N)    // disable reset
    reg.Clear(hw.ctrl, CTRL_BEE_ENABLE) // disable
}

func (hw *BEE) generateKey() (err error) {
    // avoid key exposure to external RAM
    key, err := dma.NewRegion(uint(hw.key), aes.BlockSize, false)

    if err != nil {
        return
    }

    addr, buf := key.Reserve(aes.BlockSize, 0)

    if n, err := rand.Read(buf); n != aes.BlockSize || err != nil {
        return errors.New("could not set random key")
    }

    if addr != uint(hw.key) {
        return errors.New("invalid key address")
    }

    return
}
```

Example: encrypted RAM init

Go's `unsafe` can be easily identified to spot areas that require care (e.g. pointer arithmetic), it is currently used only in register and DMA memory manipulation primitives.

There are overall only 3 occurrences of `unsafe` used in `dma` and `reg` packages.

Applications are never required to use any `unsafe` function.

No language changes mean that goroutines, channels, mutexes, interfaces can be used as usual.



type Region

Region represents a memory region allocated for DMA purposes.

func NewRegion

```
func NewRegion(addr uint, size int, unsafe bool) (r *Region, err error)
```

NewRegion initializes a memory region for DMA buffer allocation.

To avoid unforeseen consequences the caller must ensure that allocated regions do not overlap among themselves or with the global one (see Init()).

To allow allocation of DMA buffers within Go runtime memory the unsafe flag must be set.

func (*Region) Alloc

```
func (r *Region) Alloc(buf []byte, align int) (addr uint)
```

Alloc reserves a memory region for DMA purposes, copying over a buffer and returning its allocation address, with optional alignment. The region can be freed up with Free().

If the argument is a buffer previously created with Reserve(), then its address is return without any re-allocation.

The optional alignment must be a power of 2 and word alignment is always enforced, 0 means 4 on 32-bit platforms and 8 on 64-bit ones.

type BEE

BEE represents the Bus Encryption Engine instance.

func (*BEE) Enable

```
func (hw *BEE) Enable(region0 uint32, region1 uint32) (err error)
```

Enable activates the BEE using the argument memory regions, each can be up to AliasRegionSize (512 MB) in size.

After activation the regions are encrypted using AES CTR. On secure booted systems the internal OTPMK is used as key, otherwise a random one is generated and assigned.

After enabling, both regions should only be accessed through their respective aliased spaces (see AliasRegion0 and AliasRegion1) and only with caching enabled (see arm.ConfigureMMU).

func (*BEE) Init

```
func (hw *BEE) Init()
```

Init initializes the BEE module.

func (*BEE) Lock

```
func (hw *BEE) Lock()
```

Lock restricts BEE registers writing.

```
import (
    "github.com/usbarmory/tamago/soc/nxp/bee"
    "github.com/usbarmory/tamago/soc/nxp/imx6ul"
)

// Encrypt 1GB of external RAM, this is the maximum extent either
// covered by the BEE or available on USB armory Mk II boards.
region0 := uint32(imx6ul.MMDC_BASE)
region1 := region0 + bee.AliasRegionSize

imx6ul.BEE.Init()
defer imx6ul.BEE.Lock()

if err := imx6ul.BEE.Enable(region0, region1); err != nil {
    log.Fatalf("could not activate BEE: %v", err)
}
```

type CAAM

CAAM represents the Cryptographic Acceleration and Assurance Module instance.

func (*CAAM) DeriveKey

```
func (hw *CAAM) DeriveKey(versifier []byte, key []byte) (err error)
```

DeriveKey derives a hardware unique key in a manner equivalent to NXP Symmetric key diversifications guidelines (AN10922 - Rev. 2.2) for AES-256 keys.

The diversifier is used as message for AES-256-CMAC authentication using a blob key encryption key (BKEK) derived from the hardware unique key (internal OTPMK, when SNVS is enabled, through Master Key Verification Blob).

WARNING: when SNVS is not enabled a default non-unique test vector is used and therefore key derivation is **unsafe**, see `snvs.Available()`.

The unencrypted BKEK is used through DeriveKeyMemory. An output key buffer previously created with DeriveKeyMemory.Reserve() can be used to avoid external RAM exposure, when placed in iRAM, as its pointer is directly passed to the CAAM without access by the Go runtime.

i.MX6ULZ driver: USB

```
func buildDTD(n int, dir int, ioc bool, addr uint32, size int) (dtd *dTD) {
    dtd = &dTD{}

    // interrupt on completion (ioc)
    bits.SetTo(&dtd.Token, 15, ioc)

    // invalidate next pointer
    dtd.Next = 0b1
    // multiplier override (Mult0)
    bits.SetN(&dtd.Token, 10, 0b11, 0)
    // active status
    bits.Set(&dtd.Token, 7)
    // total bytes
    bits.SetN(&dtd.Token, 16, 0xffff, uint32(size))

    dtd._buf = addr
    dtd._size = uint32(size)

    for n := 0; n < DTD_PAGES; n++ {
        dtd.Buffer[n] = dtd._buf + uint32(DTD_PAGE_SIZE*n)
    }

    buf := new(bytes.Buffer)
    binary.Write(buf, binary.LittleEndian, dtd)
    dtd._dtd = dma.Alloc(buf.Bytes()[0:DTD_SIZE], DTD_ALIGN)

    return
}
```

Example of Endpoint Transfer Descriptor (dTD) configuration.

A custom DMA allocator is used to copy structures on memory reserved for DMA operation, with required alignments.

```
addr = dma.Alloc(buf, align)
defer dma.Free(addr)
```

Buffers can be also reserved by the application to spare re-allocation (automatic detection of slices already in DMA memory).

Using Go goroutines, channels, mutexes, interfaces freely in low level drivers is a delight!

All in ~1200 LOC !



i.MX6ULZ driver: Random Number Generator

The RNG provides a hardware True Random Number Generator, useful to gather the initial seed on embedded systems without a battery backed RTC (and not much else¹). The driver takes ~80 LOC and is hooked as provider for crypto/rand.

```
var getRandomDataFn func([]byte)

//go:linkname getRandomData runtime.getRandomData
func getRandomData(b []byte) {
    getRandomDataFn(b)
}

func (hw *rngb) getRandomData(b []byte) {
    read := 0
    need := len(b)

    for read < need {
        if reg.Get(hw.status, HW_RNG_SR_ERR, 0x1) != 0 {
            panic("imx6_rng: panic\n")
        }

        if reg.Get(hw.status, HW_RNG_SR_FIFO_LVL, 0xf) > 0 {
            val := *hw.fifo
            read = fill(b, read, val)
        }
    }
}
```

```
for i := 0; i < 10; i++ {
    rng := make([]byte, size)
    rand.Read(rng)
    fmt.Printf("%x\n", rng)
}
```

```
-- rng -----
imx6_rng: self-test
imx6_rng: seeding
f90b00053a50b9edd42df027c982769d1a7d25445e31ce98486bd4a9676bef42
56baf6ecc32bf02fb9d09c2d8c607baa487e2283b6856486b42cdf954277d4d5
49fc0c03f8cbc45f7aeb58ba71c0d561a91dbeae697d7bc511482697bf96b2f8
345db47ab3395272a9db9531f03160b3e1654b7e8b7267c1a3bc97206f3cb8c7
cb54154b105a2bd3938fbd99ff12f5409c0be09dc5f64189f473ae905d264b25
275994ee93e0c779f3eb30d770eeabfcb5ab0b8a5da68cc28a07dfbdb46a1e08
6215cc716b9ed577d3c6cd34d57f2dc3ed93c9b6aaedf120d68a4532393e1056
d691d7f93c57a54462f90ca76528beec4bda1a40220e5d5fbf43986308f9013b
6ea213b27eb3e0e4243b3c872e7a07b7898d9f07ea205b8a50c30e62c7204602
4544d5dff957471972331532aaf34eb5644bc430f854dd6593177640e07e4f00
```

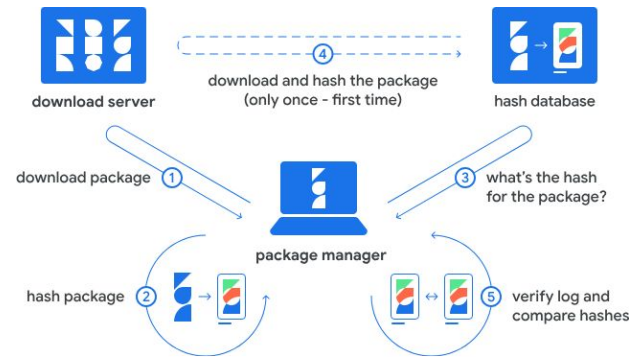
USB armory Mk II example TRNG run

¹ https://media.ccc.de/v/32c3-7441-the_plain_simple_reality_of_entropy



Reducing the attack surface

Package	LOCs	Driver support
MK2	680	USB armory Mk II board support package
ARM	900	MMU, timer, exceptions, IRQ handling
BEE	130	OTF AES RAM encryption/decryption
CAAM	840	accel. AES/ECC/CMAC/SHA/TRNG, HUK derivation
DCP	450	accel. AES/SHA, HUK derivation
ENET	400	10/100-Mbps Ethernet driver, MII support
RNGB	80	True Random Number Generator
RPMB	230	Replay Protected Memory Block
RTIC	90	Run Time Integrity Checker
SNVS	180	tamper proof sensors
USB	1200	USB 2.0 in device mode
USDHC	1100	eMMC (HS200 speed) / SD (SDR104 speed)
AMD64	600	MMU, SMP, exceptions, IRQ handling
MICROVM	150	Firecracker/QEMU microVM board support packages
UEFI	100	UEFI x64 support package
IOAPIC	60	Interrupt Controller
PCI	200	Peripheral Component Interconnect
RTC	50	Real Time Clock
CLOCK	30	KVM Clock Pairing
PVCLOCK	80	KVM PV Clock
VIRTIO	600	VirtIO over MMIO/PCI



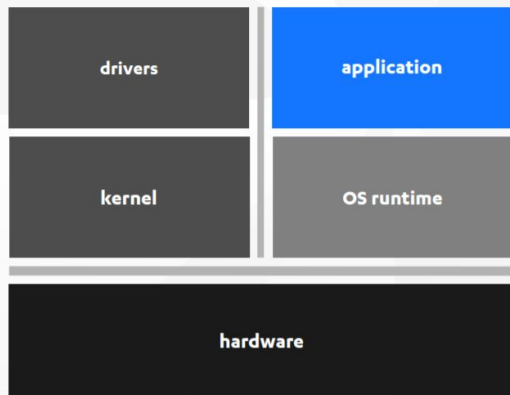
```
module github.com/usbarmory/GoTEE                                go.mod

go 1.26.0

require github.com/usbarmory/tamago v1.26.0
```

TamaGo allows creation of true unikernels, incorporating in a single binary boot code, peripheral drivers, libraries and application code with minimal dependencies and all the benefits of the full Go ecosystem, including supply chain security, build reproducibility and debugging.

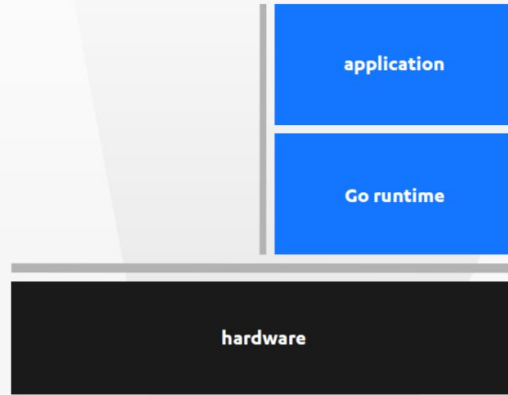
Improving memory safety



Traditional OS

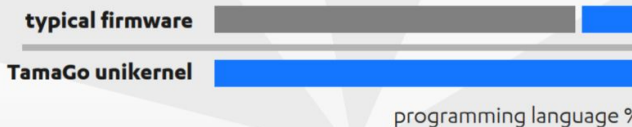


■ memory-unsafe programming language
■ memory-safe programming language



TamaGo unikernel

TamaGo allows a dramatic reduction of the attack surface by **removing any dependency on memory-unsafe languages** (e.g. C), Operating Systems and third party libraries.



Developing, building and running

The full Go runtime is supported¹ without any specific changes required on the application side (Rust on bare metal², for comparison, requires `#![no_std] pragma`).

```
package main

import (
    _ "github.com/usbarmory/tamago/board/usbarmory/mk2"
)

func main() {
    // your code
}
```

```
GOOS=tamago GOARM=7 GOARCH=arm GOOSPKG=github.com/usbarmory/tamago
${TAMAGO} build -ldflags "-T 0x80010000 -R 0x1000" main.go
```

All Go ecosystem features in terms of build reproducibility, dependency management, profiling, debugging, remain intact.

Firmware/unikernels can be compiled just as easily on Linux, Windows, macOS.

1. The application requires a single import for the board package to enable necessary initializations.
2. Go code can be written with very few limitations and the SoC package exposes driver APIs.
3. `go build` can be used as usual (reproducible builds!) with few linker flags to define entry point.
4. SoC and KVM packages support native loading (no bootloader!).

¹ <https://github.com/usbarmory/tamago/wiki/Compatibility>

² <https://rust-embedded.github.io/book/intro/no-std.html>



Performance

Two choices of pure Go TCP/IP stacks are, available, **easily switched** across through a single argument using our go-net package.

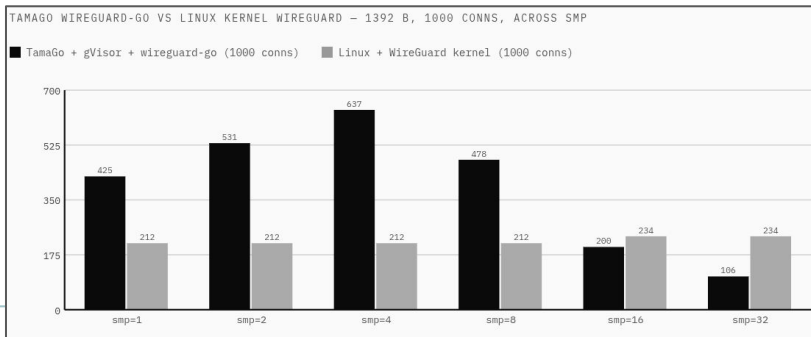
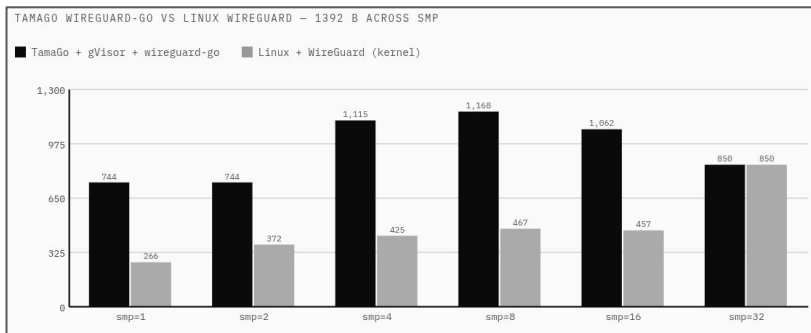
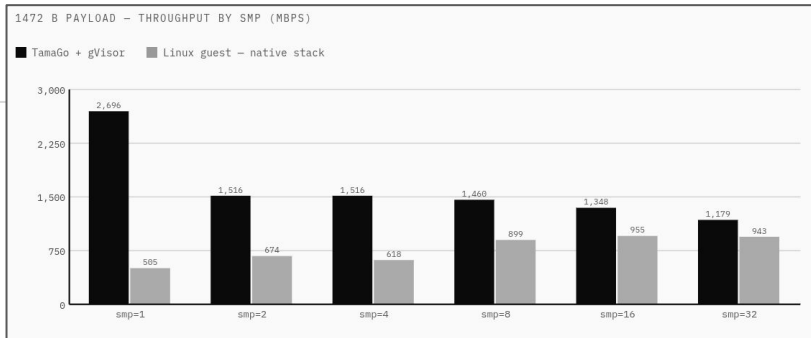
`gvisor.dev/gvisor/pkg/tcpip`
`gvisor.dev/soypat/lneto`

A TamaGo + gVisor VM **outperforms** a Linux VM (native or gVisor) by a ratio range of **50% - 500%** depending on packet size and core count.

This has been tested with simple echo servers as well as through Wireguard peering, using wireguard-go on TamaGo and both kernel and wireguard-go on Linux.



<https://github.com/usbarmory/usbarmory/go-net>



Interrupts

```
func ServiceInterrupts(isr func(id int)) {  
    c := make(chan os.Signal, 1)  
    signal.Notify(c, irqSignal) // e.g. syscall.SIGTRAP  
  
    for {  
        // re-enable interrupts after we get parked  
        go irq_enable()  
  
        // wait for interrupt raised from Go assembly  
        // handler using signal.Relay(irqSignal)  
        <-c  
  
        // service interrupt  
        isr()  
    }  
}
```

Interrupt handling is implemented by piercing the timer of a parked goroutine from Go assembly in the low-level exception handler.

The implementation is exposed through Go standard `os/signal` library.

```
runtime.Idle = func(pollUntil int64) {  
    if pollUntil == 0 {  
        return  
    }  
  
    cpu.SetAlarm(pollUntil)  
    cpu.WaitInterrupt()  
    cpu.SetAlarm(0)  
}
```

With interrupt support, CPU idle time management can be optimized.

The CPU can be halted with precision, to reduce power/resource consumption, when the runtime has nothing else but to wait for an event (e.g. network packet).

Interrupt support also enables orchestration on platforms supporting SMP (e.g. amd64).

```
$ make qemu

tamago/arm (go1.22.1) • 5e1f6aa lcars@lambda on 2024-03-20 08:02:11 • i.MX6UL 1188 MHz (emulated)

ble                                     # BLE serial console
date                                 (time in RFC339 format)?      # show/change runtime date and time
dcp                                 <size> <sec>                # benchmark hardware encryption
dns                                 <fqdn>                      # resolve domain (requires routing)
exit, quit                          # close session
help                                # this help
i2c                                 <n> <hex target> <hex addr> <size> # I²C bus read
info                                # device information
kem                                 # benchmark post-quantum KEM
led                                 # LED control
md                                 <hex offset> <size>          # memory display (use with caution)
mmc                                 <n> <hex offset> <size>      # MMC/SD card read
mw                                 <hex offset> <hex value>     # memory write (use with caution)
ntp                                <host>                      # change runtime date and time w/ NTP
otp                                <bank> <word>               # OTP fuses display
rand                               # gather 32 random bytes
reboot                             # reset device
stack                              # stack trace of current goroutine
stackall                           # stack trace of all goroutines
test                               # launch tests

> kem
Kyber1024 89248f2f33f7f4f7051729111f3849c409a933ec904aedadf035f30fa5646cd5 (287.799024ms)
Kyber768  a1e122cad3c24bc51622e4c242d8b8acbcd3f618f6e4220400605ca8f9ea02c2 (289.114896ms)
Kyber512  e9c2bd37133fcb40772f81559f14b1f58dccc1c816701be9ba6214d43baf4547 (149.049056ms)

> rand
db7d46647880be1e51731177b6f73645b71ca504242c97758df3a8684d293236

> md 80000000 96
00000000 18 f0 9f e5 18 f0 9f e5 18 f0 9f e5 18 f0 9f e5 | .....|
00000010 18 f0 9f e5 18 f0 9f e5 18 f0 9f e5 18 f0 9f e5 | .....|
00000020 04 d4 0f 80 38 d4 0f 80 6c d4 0f 80 a0 d4 0f 80 | ....8...l.....|
00000030 d4 d4 0f 80 00 00 00 00 08 d5 0f 80 3c d5 0f 80 | .....<.....|
00000040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....|
00000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....|

> ntp time.google.com
2024-03-20T08:02:11Z

> info
Runtime .....: go1.22.1 tamago/arm
RAM .....: 0x80000000-0x9f600000 (502 MiB)
Board .....: UA-MKII-y
SoC .....: i.MX6ULZ 900 MHz
SSM Status ...: state:0b1101 clk:false tmp:false vcc:false hac:4294967295
Boot ROM hash : 1727a0f46dbde555b583e9a138ae359389974b7be4369ffd4a252a8730f7e59b
Secure boot ..: true
Unique ID ....: FE186D5AB312430B
SDP .....: true
Temperature ...: 48.333332
```

```
$ ssh 10.0.0.1

tamago/arm (go1.22.1) • 5e1f6aa lcars@lambda on 2024-03-20 10:00:48 • i.MX6ULL 900 MHz

> otp 0 0
OTP bank:0 word:0 val:0x00324003

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 09 08 07 06 05 04 03 02 01 00 OCOTP_LOCK
|0|0|0| | | | |0| |0|0|1|1|0|0|1|0|0|1|0|0|0|0|0|0| |0|0|0|0|0|1|1|R:
0x00000000
W: 0x00000000

31
30 ----- GP3_LOCK
25 ----- GP4_LOCK
23 ----- PIN_LOCK
22 ----- GP4_LOCK
21 ----- MISC_CONF_LOCK
20 ----- ROM_PATCH_LOCK
19 18 ----- OTPMK_CRC_LOCK
17 ----- ANALOG_LOCK
16 ----- OTPMK_LOCK
15 ----- SW_GP_LOCK
14 ----- GP3_LOCK
13 12 ----- SRK_LOCK
11 10 ----- GP2_LOCK
09 08 ----- GP1_LOCK
06 ----- MAC_ADDR_LOCK
05 04 ----- SJC_RESP_LOCK
03 02 ----- MEM_TRIM_LOCK
01 00 ----- BOOT_CFG_LOCK
TESTER_LOCK
```

```
> dns www.golang.org
[142.251.215.238 2607:f8b0:400a:805::200e]

> dcp 65536 10
Doing aes-128 cbc for 10s on 65536 blocks
6201 aes-128 cbc's in 10.00086575s

> info
Runtime .....: go1.22.1 tamago/arm
RAM .....: 0x80000000-0x9f600000 (502 MiB)
Board .....: UA-MKII-y
SoC .....: i.MX6ULZ 900 MHz
SSM Status ...: state:0b1101 clk:false tmp:false vcc:false hac:4294967295
Boot ROM hash : 1727a0f46dbde555b583e9a138ae359389974b7be4369ffd4a252a8730f7e59b
Secure boot ..: true
Unique ID ....: FE186D5AB312430B
SDP .....: true
Temperature ...: 48.333332
```

[illegible]

Machine Details

Information about this machine's network. Used to debug connection issues.

Creator	[REDACTED]	Created	Jun 15, 2023 at 1:06 PM GMT+2
Machine name	tamago Copy	Last seen	1:06 PM GMT+2
OS hostname	tamago	Key expiry	6 months from now
OS	Tamago		
Tailscale version	1.38.4-dev20230612	CLIENT CONNECTIVITY	
Tailscale IPv4	100 [REDACTED] Copy	Varies	☐
Tailscale IPv6	fd76 [REDACTED] Copy	Hairpinning	☐
ID	nt4 [REDACTED]	IPv6	☐
Endpoints	—	UDP	☐
Relays	—	UPnP	☐
		PCP	☐
		NAT-PMP	☐

```
$ ping 100.xxx.xx.82
```

```

PING 100.xxx.xx.82 (100.xxx.xx.82) 56(84) bytes of data.
64 bytes from 100.xxx.xx.82: icmp_seq=1 ttl=64 time=170 ms
64 bytes from 100.xxx.xx.82: icmp_seq=2 ttl=64 time=161 ms
64 bytes from 100.xxx.xx.82: icmp_seq=3 ttl=64 time=126 ms

```

GoKey - The bare metal Go smart card

The GoKey application implements a composite USB device implementing an **OpenPGP 3.4** smartcard, a **FIDO U2F** token, an **age** plugin and a **PKCS#11 over RPC** server, written in pure Go (~2700¹ LOC).

It allows to implement a radically different security model for smartcards, taking advantage of Go to safely mix layers and protocols not easy to combine.

For instance authentication can happen over SSH instead of plaintext PIN transmission over USB.

	Trust anchor	Data protection	Runtime	Application	Requires tamper proofing	Encryption at rest
traditional smartcard	flash protection	flash protection	JCOP	JCOP applets	Yes	No
USB armory with GoKey	secure boot	SoC security element	TamaGo	Go application	No	Yes

```
GoKey
host > gpg --card-status
Reader .....: USB armory Mk II [Smart Card Control] (0.1) 00 00
Application ID ...: D27600124010304F5ECD209320C0000
Application type ..: OpenPGP
Version .....: 3.4
Manufacturer .....: F-Secure
Serial number ....: D209320C
Name of cardholder: Alice
Language prefs ...: [not set]
Salutation .....:
URL of public key : [not set]
Login data .....: [not set]
Signature PIN ....: forced
Key attributes ...: rsa4096 rsa4096 rsa4096
Max. PIN lengths ..: 254 127 127
PIN retry counter ..: 1 0 0
Signature counter ..: 0
Signature key ....: 05EC DEB4 43FA 5C01 9C7A 51A2 E9C8 5194 3E46 C2B5
created .....: 2020-04-03 15:10:30
Encryption key ....: 656B E354 EE12 BFFB 988B 1607 556B 9659 5A2C D776
created .....: 2020-04-03 15:01:49
Authentication key: [none]
General key info...: sub rsa4096/E9C851943E46C2B5 2020-04-03 Alice <alice@wonderl
and>
sec# rsa4096/CBBE74C25E15EA0B created: 2020-04-03 expires: 2022-04-03
ssb# rsa4096/556B96595A2CD776 created: 2020-04-03 expires: 2022-04-03
card-no: F5EC D209320C
ssb# rsa4096/E9C851943E46C2B5 created: 2020-04-03 expires: 2022-04-03
card-no: F5EC D209320C
ssb# rsa4096/2EB31B5E996EE830 created: 2020-04-03 expires: 2022-04-03
host > ssh alice@10.0.0.10
GoKey > tamago/arm (gol.14) > 0330e82 user@host on 2020-04-09 07:42:11 > 1.MXGULL

exit, quit # close session
help # this help
init # initialize card
rand # gather 32 bytes from TRNG via crypto/rand
reboot # restart
status # display card status
lock # key lock
unlock (all|sig|dec) # key unlock, prompts decryption passphrase

resizing terminal (pty-req:80x66)
> unlock all
Passphrase:
VERIFY: 05 EC DE B4 43 FA 5C 01 9C 7A 51 A2 E9 C8 51 94 3E 46 C2 B5 unlocked
VERIFY: 65 6B E3 54 EE 12 BF FB 98 8B 16 07 55 6B 96 59 5A 2C D7 76 unlocked

> exit
logout
closing ssh connection
Connection to 10.0.0.10 closed.
host > gpg --decrypt secret.asc
gpg: encrypted with 4096-bit RSA key, ID 556B96595A2CD776, created 2020-04-03
cheshire wrote:
"Alice <alice@wonderland>
"Where do you want to go?"

alice wrote:
"I don't know"

cheshire wrote:
"Then, it really doesn't matter, does it?"
host >
```





GoKey - the bare metal Go smart card





Armory Drive - Encrypted USB Mass Storage

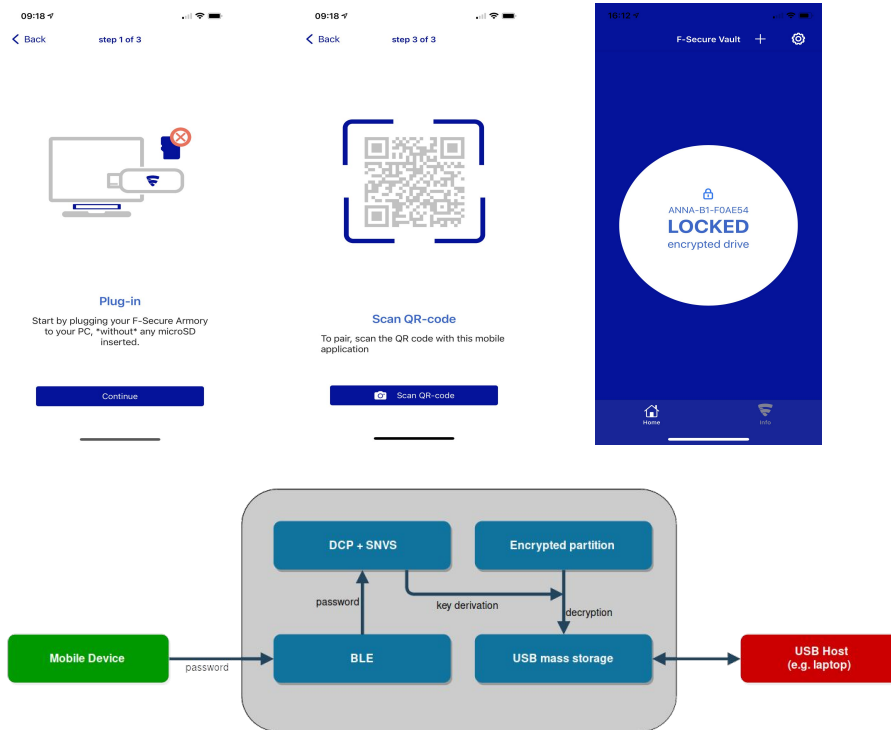
Armory Drive implements the **easiest to use encrypted drive solution** allowing secure access to any microSD card.

Unlike existing encrypted drive solutions the key is unlocked with **3 factors** (user + mobile phone + armory) and **over Bluetooth**. No trust (or driver requirements) are delegated to the host.

It consists of **~3000 LOC** of pure TamaGo code and an iOS app.



<https://github.com/usbarmory/armory-drive>



It uses Google Firmware Transparency framework to enable firmware update authentication on the installer as well as the device itself.

<https://youtu.be/watch?v=Sv2wdVkp03A>

armory-boot - USB armory boot loader

A primary signed boot loader (~700 LOC) to launch authenticated Linux kernel images on secure booted¹ USB armory boards, replacing U-Boot.

```
func boot(kernel uint, params uint, cleanup func(), region *dma.Region) (err error) {
    table := arm.SystemVectorTable()
    table.Supervisor = exec

    imx6ul.ARM.SetVectorTable(table)

    _kernel = uint32(kernel)
    _params = uint32(params)
    _mmu = (region != nil)

    cleanup()

    if region != nil {
        imx6ul.ARM.SetAttribute(
            uint32(region.Start()),
            uint32(region.End()),
            arm.TTE_EXECUTE_NEVER, 0)
    } else {
        imx6ul.ARM.FlushDataCache()
        imx6ul.ARM.DisableCache()
    }

    svc()

    return errors.New("supervisor failure")
}
```

```
func verifySignature(buf []byte, s []byte) (valid bool, err error) {
    sig, err := DecodeSignature(string(s))

    if err != nil {
        return false, fmt.Errorf("invalid signature, %v", err)
    }

    pub, err := NewPublicKey(PublicKeyStr)

    if err != nil {
        return false, fmt.Errorf("invalid public key, %v", err)
    }

    return pub.Verify(buf, sig)
}

func verifyHash(buf []byte, s string) bool {
    // use hardware acceleration
    sum, _ := imx6ul.DCP.Sum256(buf) {

        if hash, err := hex.DecodeString(s); err != nil {
            return false
        }

        return bytes.Equal(sum[:], hash)
    }
}
```



type ELFImage

```
type ELFImage struct {  
    // Region is the memory area for image loading.  
    Region *dma.Region  
    // ELF is a bootable bare-metal ELF image.  
    ELF []byte  
    // contains filtered or unexported fields  
}
```

func (*ELFImage) Boot

```
func (image *ELFImage) Boot(cleanup func()) (err error)
```

Boot calls a loaded bare-metal ELF image.

func (*ELFImage) Entry

```
func (image *ELFImage) Entry() uint
```

Entry returns the image entry address.

func (*ELFImage) Load

```
func (image *ELFImage) Load() (err error)
```

Load loads a bare-metal ELF image in memory.

ELFImage represents a bootable bare-metal ELF image.

```
import (  
    "github.com/usbarmory/armory-boot/exec"  
)  
  
image := &exec.ELFImage{  
    Region: mem,  
    ELF:    os.Firmware,  
}  
  
if err = image.Load(); err != nil {  
    panic(fmt.Sprintf("load error, %v\n", err))  
}  
  
log.Printf("armored-witness-boot: starting kernel@%.8x\n", image.Entry())  
  
if err = image.Boot(preLaunch); err != nil {  
    panic(fmt.Sprintf("armored-witness-boot: load error, %v\n", err))  
}
```

go-boot - UEFI boot manager

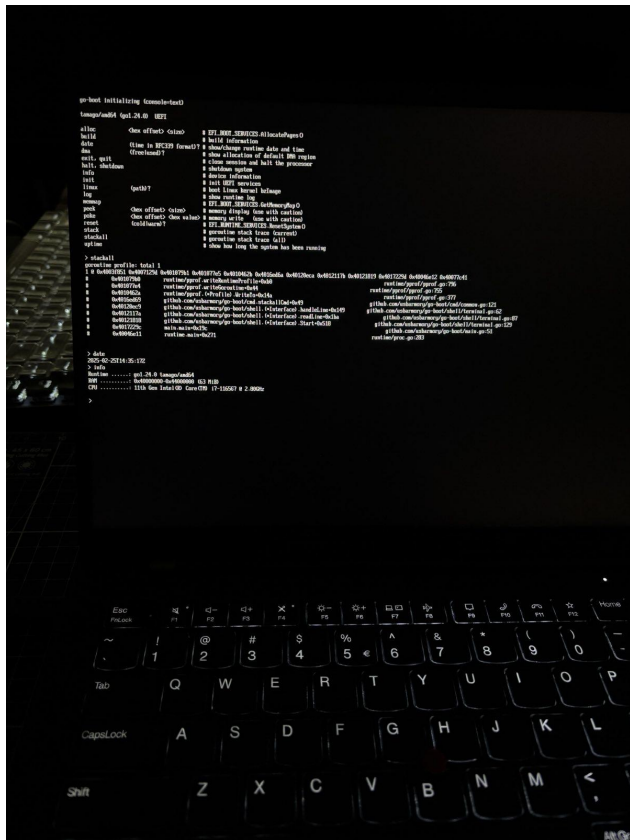
A TamaGo unikernel implementing a **UEFI Shell** and **OS loader** for AMD64 platforms.

It supports loading of arbitrary EFI images, Linux kernels and Windows UEFI boot manager.

It provides a clean pure Go API to UEFI Boot and Runtime services in ~2000 LOC.

It results in a 3MB EFI application which performs as fast, if not faster, than equivalent functionality Implemented in other C/Rust based efforts.

It supports **UEFI networking** through SNP, bringing full Go network I/O in your pre-boot environment.



type Services

Services represents the UEFI services instance.

```
type Services struct {  
    // EFI System Table instance  
    SystemTable *SystemTable  
  
    // UEFI services  
    Console *Console  
    Boot    *BootServices  
    Runtime *RuntimeServices  
    // contains filtered or unexported fields  
}
```

func (*Services) Address

```
func (s *Services) Address() uint64
```

Address returns the EFI System Table pointer.

func (*Services) Root

```
func (s *Services) Root() (root *FS, err error)
```

Root returns an EFI Simple File System instance for the current EFI image root volume.

```
import (  
    "github.com/usbarmory/go-boot/uefi"  
)  
  
UEFI := &uefi.Services{  
  
    if err = UEFI.Init(imageHandle, systemTable); err != nil {  
        panic(fmt.Sprintf("could not initialize EFI services, %v\n", err))  
    }  
  
    root, _ = UEFI.Root()  
    UEFI.Boot.LoadImage(0, root, "EFIShell.efi")  
}
```

go-boot shell

initializing EFI services

go-boot • tamago/amd64 (go1.26.4) • UEFI x64

```
.                # `.\efi\boot\bootx64.efi`
.                # load and start EFI image
build            # build information
cat              <path>          # show file contents
clear            # clear screen
cpuid            <leaf> <subleaf> # show CPU capabilities
date             (time in RFC339 format)? # show/change runtime date and time
dns              <host>          # resolve domain
exit,quit        # exit application
halt,shutdown    # shutdown system
help             # this help
info             # device information
linux,l          (loader entry path)? # boot linux kernel image
linux,l,\r       # `l \loader\entries\arch.conf`
log              # show runtime logs
ls               (path)?         # list directory contents
lspci            # list PCI devices
memmap           (e820)?         # show UEFI memory map
mode             <mode>         # set screen mode
net              <ip> <mac> <gw> (debug)? # start UEFI networking
peek            <hex offset> <size> # memory display (use with caution)
poke            <hex offset> <hex value> # memory write (use with caution)
protocol         <registry format GUID> # locate UEFI protocol
reset            (cold|warm)?    # reset system
stack            # goroutine stack trace (current)
stackall         # goroutine stack trace (all)
stat             <path>         # show file information
uefi             # UEFI information
uptime          # show system running time
windows,win,w    # launch Windows UEFI boot manager
```

> net 10.0.0.1/24 : 10.0.0.2 debug

starting debug servers:

http://10.0.0.1:80/debug/pprof ssh://10.0.0.1:22

network initialized (10.0.0.1/24 16:8c:fe:73:31:55)

> info

Runtime: go1.26.4 tamago/amd64

RAM: 0x10000000-0xbbf6d000 (2751 MiB)

Text: 0x10010000-0x1048d0f0

Heap: 0x10b2d030-0xbbf6d000 Alloc:1 MiB Sys:3 MiB

CPU: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz

Frequency: 2.8032 GHz

> uefi

UEFI Revision: 2.70

Firmware Vendor: EDK II

Firmware Revision ...: 0x10000

Runtime Services ...: 0xbf9ecb98

Boot Services: 0xbfea5720

Frame Buffer: 1280x800 @ 0xc0000000

Configuration Tables: 0xbf9ecc98

...

dcfa911d-26eb-469f-a220-38b7dc461220 (0xbebbd018)

> ls /efi/boot

d . d .. f bootx64.efi

> stat /efi/boot/bootx64.efi

Size:11487374 ModTime:2025-10-09 07:15:38 +0000 tz IsDir:false Sys:0xbe3d0da0

Sum256:a15fa478b0438dcf3896b7dfcfce47f9e2101b39ba210cb0766ba2541950487c

> cpuid 0 0

EAX EBX ECX EDX

0000001b 756e6547 6c65746e 49656e69

Less is more

Bringing bare metal Go to pre-boot UEFI applications allows overall reduction of UEFI usage, while benefiting from its boot services and runtime context.

UEFI TCPv4/TCPv6	replaced by gvisor over Simple Network Protocol
UEFI TLS, HTTP/HTTPS	replaced by Go net/tls, net/http state-of-the-art implementation
UEFI Regular Expression	replaced by Go regexp
UEFI Byte Code	replaced by wazero
UEFI HPSee	replaced by Tailscale

We spare UEFI from complex, potentially unsafe, least performant and potentially out-of-date protocols, favoring **modern, secure, application/user controlled** stacks.

The burden placed towards UEFI are reduced, without compromises, restoring a more favorable separation of roles.



UEFI Frame Buffer

+

Fyne

+

Tailscale



```
initializing EFI services
initializing console (text)
```

```
go-boot • tamago/amd64 (go1.25.1) • UEFI x64
```

```
,
,
<path> # `.\efi\boot\bootx64.efi`
# load and start EFI image
build # build information
cat # show file contents
clear # clear screen
cpuid # show CPU capabilities
date (time in RFC339 format)? # show/change runtime date and time
dns <host> # resolve domain
exit,quit # exit application
gemini # ask gemini
halt,shutdown # shutdown system
help # this help
info # device information
linux,l (loader entry path)? # boot Linux kernel image
linux,l,r # `l \loader\entries\arch.conf`
log # show runtime logs
ls (path)? # list directory contents
lspci # list PCI devices
memmap (e820)? # show UEFI memory map
mode <mode> # set screen mode
net <ip> <mac> <gw> <debug>? # start UEFI networking
peek <hex offset> <size> # memory display (use with caution)
poke <hex offset> <hex value> # memory write (use with caution)
protocol <registry format GUID> # locate UEFI protocol
reset (cold|warm)? # reset system
stack # goroutine stack trace (current)
stackall # goroutine stack trace (all)
stat <path> # show file information
uefi # UEFI information
uptime # show system running time
windows,win,w # launch Windows UEFI boot manager
```

```
> net 10.0.0.1/24 : 10.0.0.2
```

```
network initialized (10.0.0.1/24 56:6d:ab:80:04:5b)
```

```
> gemini I am talking to you from an UEFI bootloader, isn't that amazing? Shall I
```

```
boot Linux or Windows today?
```

```
That *is* amazing! It's cool to think I'm communicating with you directly from the UEFI environment.
```

As for your question, whether to boot Linux or Windows is entirely up to you! It depends on what you want to do today. Here's a little thought process to help you decide:

```
* **What tasks do you have planned?**
* **Linux:** If you're planning on doing software development, system administration, using specific Linux-only tools, or prefer its command-line environment, Linux might be the better choice.
* **Windows:** If you're planning on gaming, using Windows-specific applications (like Microsoft Office), or prefer its user interface, Windows might be the better choice.

* **What are you in the mood for?**
* Sometimes, it's just about which operating system you feel like using!
```

Ultimately, the decision is yours. Have fun with whichever OS you choose! Let me know if you need any more help or have any other questions.

```
> []
```



Support for cloud deployments

TamaGo Unikernels can be converted to raw MBR disk images to be launched as VMs on **Google Compute Engine** or **Amazon EC2**.

On compute platforms with UEFI support it is also possible to simply boot as UEFI binary.

This is supported by tamago BSP
google/gcp and go-boot BSP uefi/x64.

```
> info
Runtime .....: go1.25.5 tamago/amd64 GOMAXPROCS=1
RAM .....: 0x10000000-0x50000000 (1024 MiB)
Board .....: gcp
CPU .....: AMD EPYC 7B13
Cores .....: 1
Frequency ....: 2.451697 GHz
VirtIO Net0 ..: 42:01:0a:84:00:02
```

The screenshot shows the Google Cloud Platform console interface. On the left, the 'Compute Engine' sidebar is visible with various navigation options. The main panel displays the 'Serial port 1' output for a VM named 'ops-tamago-example-1759325667'. The output shows the SeaBIOS boot process, including memory and CPU information, and the virtio-scsi controller details. Below this, the 'tamago/amd64' kernel output is shown, displaying a list of commands and their corresponding actions, such as starting a remote file server, building information, showing file contents, displaying CPU capabilities, and more.

```
Serial port 1 (console) output for ops-tamago-example-1759325667

SeaBIOS (version 1.8.2-google)
Total RAM Size = 0x00000001e0000000 = 7680 MiB
CPUs found: 2    Max CPUs supported: 2
Found virtio-scsi at 0:3
virtio-scsi vendor='Google' product='PersistentDisk' rev='1' type=0 removable=0
virtio-scsi blksize=512 sectors=2097152 = 1024 MiB
drive 0x000f2000: PCHS=0/0/0 translation=lba LCHS=1024/32/63 s=2097152
Sending Seabios boot VM event.
Booting from Hard Disk 0...
READSECTORS RAAR-000 could not start networking, could not initialize VirtIO networking, invalid VirtIO instance

tamago/amd64 (go1.25.1) · Intel(R) Xeon(R) CPU @ 2.30GHz

[36m9p
build
cat
cpuid
date
dma
dns
exit, quit
halt
info
kem
linux
ls
ntp
peek
poke
rand
reboot
smp
stack
stackall
tailscale
test
uptime
wormhole
[0m
[31m [0m

# start 9p remote file server
# build information
# show file contents
# display CPU capabilities
# show/change runtime date and time
# show allocation of default DMA region
# resolve domain
# close session
# halt the machine
# device information
# benchmark post-quantum KEM
# boot Linux kernel bzImage
# list directory contents
# change runtime date and time via NTP
# memory display (use with caution)
# memory write (use with caution)
# gather 32 random bytes
# reset device
# launch SMP test
# goroutine stack trace (current)
# goroutine stack trace (all)
# start network servers on Tailscale tailnet
# launch tests
# show how long the system has been running
# transfer file through magic wormhole
```



TamaGo Unikernels can be launched as Confidential VMs, supporting **AMD SEV-SNP** interaction through a dedicated package.

Through `uefi/x64` and `sev` packages, attestation reports can be requested, UEFI services can be terminated for the rest of the unikernel execution if desired.

Networking is supported through UEFI Simple Network Protocol or VirtIO.

SMP can be initialized through:

```
(mp *MultiProcessor).StartupAllAPs
(ghcb *GHCB).CreateAP
```

```
> sev
SEV .....: true
SEV-ES .....: true
SEV-SNP .....: true
Encrypted bit .....: 51
SNP Version .....: 1

Secrets Page .....: 0x80d000 (4096 bytes)
Secrets Version .....: 4
TSC Factor .....: 0xc8
Launch Mitigations ..: 0xb
VMPCs .....: 0x08 ... 0x4c | 0xd7 ... 0x99 | 0x45 ... 0x86 | 0xd5 ... 0xa0
vCPU .....: 4

> sev-report
Version .....: 5
VMPL .....: 0
SignatureAlgo .....: 1
CurrentTCB .....: 1b1b00000000000a
Measurement .....: 81aee09d5c062ee8...
ReportedTCB .....: 1b1b00000000000a...
CommittedTCB .....: 1b1b00000000000a...
Launch Mitigations : 0xb
Current Mitigations : 0xb
SignatureR .....: 1e6da2bac3327aed...
SignatureS .....: 94cb3a662bd721a6...

> net-virtio 10.0.0.1/24 : 10.0.0.2 debug
starting debug servers:
  http://10.0.0.1:80/debug/pprof
  ssh://10.0.0.1:22
network initialized (10.0.0.1/24 4a:4a:05:1a:23:40)
```

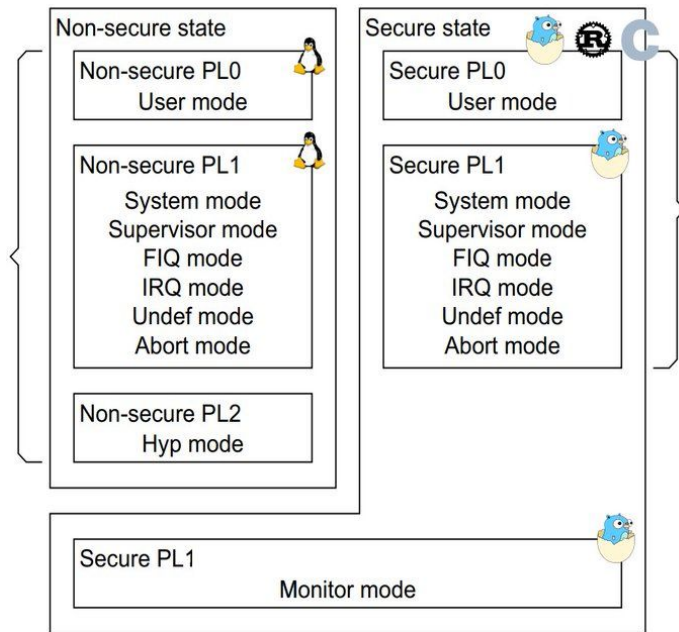


GoTEE - Trusted Execution Environment

The GoTEE framework implements concurrent instantiation of TamaGo based unikernels in privileged and unprivileged modes, interacting with each other through monitor mode and custom system calls.

With these capabilities GoTEE implements a pure Go Trusted Execution Environment (TEE) bringing Go memory safety, convenience and capabilities to bare metal execution within TrustZone Secure World.

It supports any freestanding user mode applets (e.g. TamaGo, C, Rust) and any “rich” OS running in NonSecure World (e.g. Linux).



Package monitor

```
type ExecCtx struct {
    R0  uint32
    ...
    R15 uint32 // PC

    // Memory is the executable allocated RAM
    Memory *dma.Region
    // Handler, if not nil, handles user syscalls
    Handler func(ctx *ExecCtx) error
    // Server, if not nil, serves RPC calls over syscalls
    Server *rpc.Server
}
```

func Load

```
func Load(entry uint, mem *dma.Region, secure bool) (ctx *ExecCtx, err error)
```

Load returns an execution context initialized for the argument entry point and memory region, the secure flag controls whether the context belongs to a secure partition (e.g. TrustZone Secure World) or a non-secure one (e.g. TrustZone Normal World).

func (*ExecCtx) Run

```
func (ctx *ExecCtx) Run() (err error)
```

Run starts the execution context and handles system or monitor calls. The execution yields back to the invoking Go runtime only when exceptions are caught. The function invokes the context Handler() and returns when an unhandled exception, or any other error, is raised.

```
import (
    "github.com/usbarmory/armory-boot/exec"
    "github.com/usbarmory/GoTEE/monitor"
)

image := &exec.ELFImage{
    Region: appletRegion,
    ELF:    elf,
}

if err = image.Load(); err != nil {
    return
}

if ta, err = monitor.Load(image.Entry(), image.Region, true); err != nil {
    return nil, fmt.Errorf("SM could not load applet: %v", err)
}

ta.Handler = handler
ta.Server.Register(ctl.RPC)
ta.Run()
```

```

> gotee
PL1 tamago/arm (go1.18.3) • TEE system/monitor (Secure World)
PL1 loaded applet addr:0x9c000000 size:4719809 entry:0x9c06f188
PL1 loaded kernel addr:0x80000000 size:4384184 entry:0x8006db70
PL1 starting mode:SYS ns:true sp:0x00000000 pc:0x8006db70
PL1 starting mode:USR ns:false sp:0x9e000000 pc:0x9c06f188
PL1 tamago/arm (go1.18.4) • system/supervisor (Normal World)
PL1 in Normal World is about to perform DCP key derivation
PL1 in Normal World successfully used DCP (df3eed2a50c9dd22daf7cf864f27bb90)
PL1 in Normal World is about to yield back
    r0:00000000 r1:814243f0 r2:00000001 r3:00000000
    r4:00000000 r5:00000000 r6:00000000 r7:8146bf14
    r8:00000007 r9:00000034 r10:814040f0 r11:802e9b21 cpsr:600001d6 (MON)
    r12:00000000 sp:8146bf54 lr:80185518 pc:80185648 spsr:600001df (SYS)
PL1 stopped mode:SYS ns:true sp:0x8146bf54 lr:0x80185518 pc:0x80185648 err:exit
PL0 tamago/arm (go1.18.4) • TEE user applet (Secure World)
PL0 obtained 16 random bytes from PL1: 10e742f0dad15db3f00aea14ee4a5acc
PL1 loaded kernel addr:0x80000000 size:4384184 entry:0x8006db70
PL1 re-launching kernel with TrustZone restrictions
PL1 starting mode:SYS ns:true sp:0x00000000 pc:0x8006db70
PL1 tamago/arm (go1.18.4) • system/supervisor (Normal World)
PL1 in Normal World is about to perform DCP key derivation
    r0:02280000 r1:814683a0 r2:8143c588 r3:00000001
    r4:00000000 r5:00000000 r6:00000000 r7:8146bf14
    r8:00000007 r9:00000044 r10:814040f0 r11:802e9b21 cpsr:200001d6 (MON)
    r12:00000000 sp:8146bf28 lr:80180398 pc:80011340 spsr:200001df (SYS)
PL1 stopped mode:SYS ns:true sp:0x8146bf28 lr:0x80180398 pc:0x80011340 err:DATA_ABORT
PL1 in Secure World is about to perform DCP key derivation
PL1 in Secure World World successfully used DCP (df3eed2a50c9dd22daf7cf864f27bb90)

```

```
$ ssh 10.0.0.1
```

```
PL1 tamago/arm (go1.18.3) • TEE system/monitor (Secure World)
```

```

help                # this help
reboot              # reset the SoC/board
stack               # stack trace of current goroutine
stackall            # stack trace of all goroutines
md <hex offset> <size> # memory display (use with caution)
mw <hex offset> <hex value> # memory write (use with caution)

```

```

gotee                # TrustZone test w/ TamaGo unikernels
linux <uSD|eMMC>      # boot NonSecure USB armory Debian image
lockstep <% fault>    # tandem applet example w/ fault injection

```

```

dbg                  # show ARM debug permissions
csl                  # show config security levels (CSL)
csl <periph> <slave> <hex csl> # set config security level (CSL)
sa                   # show security access (SA)
sa <id> <secure|nonsecure> # set security access (SA)

```

```

> dbg
| type                | implemented | enabled |
|-----|-----|-----|
| Secure non-invasive |           1 |         0 |
| Secure invasive    |           1 |         0 |
| Non-secure non-invasive |           1 |         1 |
| Non-secure invasive |           1 |         0 |

```

```

> linux eMMC
armory-boot: loading configuration at /boot/armory-boot-nonsecure.conf
PL1 loaded kernel addr:0x80000000 size:7603616 entry:0x80080000
PL1 launching Linux
PL1 starting mode:SVC ns:true sp:0x00000000 pc:0x80080000
Booting Linux on physical CPU 0x0
Linux version 5.15.52-0 (usbarmory@usbarmory) arm-linux-gnueabi-hf-gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1)

```

Opportunistic soft lockstep

```
> lockstep 0.01
SM loading applet in lockstep shadow memory
SM loaded applet addr:0x10000000 entry:0x10082658 size:5002440
SM starting mode:USR sp:0x12000000 pc:0x10082658 ns:false
tamago/arm • TEE user applet
applet obtained 16 random bytes from monitor: 2c4337ed2da3d3570d7120d7d52dbda5
applet requests echo via RPC: hello
applet received echo via RPC: hello
applet will sleep for 5 seconds
applet says 1 mississippi
applet says 2 mississippi
!! injecting register fault !!
SM stopped mode:USR sp:0x11ffffd0 lr:0x101acbb4 pc:0x101acbc0 ns:false err:lockstep failure
    r0:00000002 r1:11ff02a0 r2:00000000 r3:00000002
    r4:00000000 r5:00000000 r6:00000000 r7:00000001
    r8:00000001 r9:10365178 r10:10363e00 r11:00000007 cpsr:200001d3 (SVC)
    r12:584875b0 sp:11ffffd0 lr:101acbb4 pc:101acbc0 spsr:200001d0 (USR)
shadow context:
    r0:00000003 r1:11ff02a0 r2:00000000 r3:00000002
    r4:00000000 r5:00000000 r6:00000000 r7:00000001
    r8:00000001 r9:10365178 r10:10363e00 r11:00000007 cpsr:200001d3 (SVC)
    r12:584875b0 sp:11ffffd0 lr:101acbb4 pc:101acbc0 spsr:200001d0 (USR)
stack trace:
github.com/usbarmory/GoTEE/syscall/syscall_arm.s:37
github.com/usbarmory/GoTEE/applet/nanotime_syscall.go:21

>
```

GoTEE supports **lockstep execution** of any deterministic execution context, such as a Trusted Applet.

The tandem progression of a Trusted Applet executable allows detection of injected or naturally occurred faults.

To achieve this a shadow execution context is paired to the primary one for comparison at each monitor call.

All it takes is the definition of a function that re-configures virtual memory addressing so that the same applet can be allocated in two separate physical memory areas.

```
ta, _ := monitor.Load(taEntry, taMemory, true)
ta.Shadow = ta.Clone()

flags := arm.MemoryRegion|arm.TTE_AP_011<<10

ta.MMU = func() {
    imx6ul.ConfigureMMU(start, end, primaryStart, flags)
}

ta.Shadow.MMU = func() {
    imx6ul.ConfigureMMU(start, end, shadowStart, flags)
}
```

Execution reproducibility

TamaGo ZK Deterministic Execution

PROOF VALID

// 29 steps · created 1/1/1970, 1:00:00 AM

Machine ID

E3b0c44298fc1c149afbf4c8...

Witness Root

4004eab980ee4e01a33947...

Created

1/1/1970, 1:00:00 AM

Transcript Root

ee69443f7c214866fc496a...

- ✓ Step count: 29 steps
- ✓ Machine commitment: e3b0c44298fc1c149a.....
- ✓ Transcript root: ee69443f7c214866fc496aa791.....
- ✓ Witness root: 4004eab980ee4e01a33947df65.....
- ✓ Step #0: Merkle path has 5 siblings, root consistent
- ✓ Step #14: Merkle path has 5 siblings, root consistent
- ✓ Step #28: Merkle path has 5 siblings, root consistent
- ✓ Witness root matches last step hash – chain intact
- ✓ PROOF VALID – execution was deterministic and untampered

Steps
29

#	Kind	Time (ns)	Data	Step Hash
000	INPUT	331,050,001	"tamago-zk-poc-input-v1"	a40b363b76ae15dea7bee0...
001	TIME	340,750,001	b16e4f1400000000	340d8bcd170e4994965335...
002	RAND	340,750,001	a8849f273f81aa75e881bf9b59e8a3...	83d39fa65836a276e090ce...
003	HASH	340,750,001	76b3e5d6979ffc8aa7d6948d66e52c...	c082a0c97b2279e7b12f50...
004	TIME	370,400,001	01db131600000000	a66f7686c9eac802a5950c...
005	RAND	370,400,001	93d8e47efc6c90fa8d2942dbb9ce59...	b612f66c8c243e5219d972...
006	HASH	370,400,001	54eb9043d1c328e6507bf0f9311fa1...	d87fcb8805d1e882ebc121...

GoTEE in the wild: space!

On May 22nd 2023 (UTC 05:00:32) the USB armory Mk II got a lift to space!

On February 27th 2024 (UTC 07:27:00) the same unit went back to space.

GoTEE supervised a TamaGo based unikernel (acting as Trusted Applet) and a full Linux instance isolated in NonSecure World to test Post Quantum Key exchanges.

For this occasion TamaGo and GoTEE have been updated with full watchdog and interrupt support.

The payload remained operation for the entire flight duration performing exactly 400 PQC key exchanges. As far as we know this is the first time bare metal Go executed in space.



Andrea Barisani
@AndreaBarisani

...

Yesterday (UTC 05:00:32) @WithSecure USB armory got a lift (~225 km apogee) from the MAPHEUS-13 rocket launched from the Esrange Space Center.

Thanks to @DLR_de @adesso_SE for this amazing collaboration!

Our bare metal GoTEE performed Post Quantum key exchange in space!



GoTEE in the wild: Armored Witness

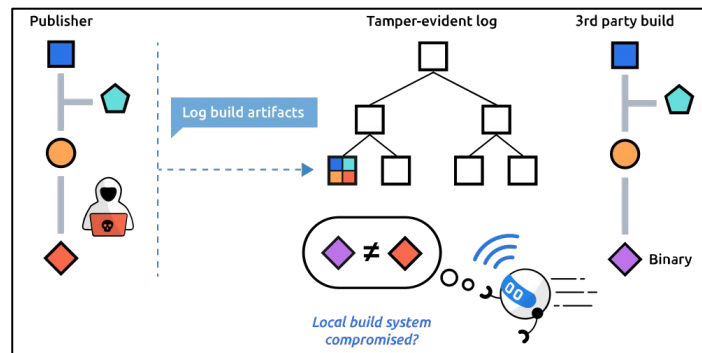
The applet observes public transparency logs verifying that they're operating in an append-only fashion, and counter-signing those checkpoints which it has determined are consistent with all previous checkpoints its seen from the same log.

The counter-signed checkpoints are sent to a **distributor**, which then collates counter-signatures for a given checkpoint from one or more Armored Witness devices, and serves them via a public API.

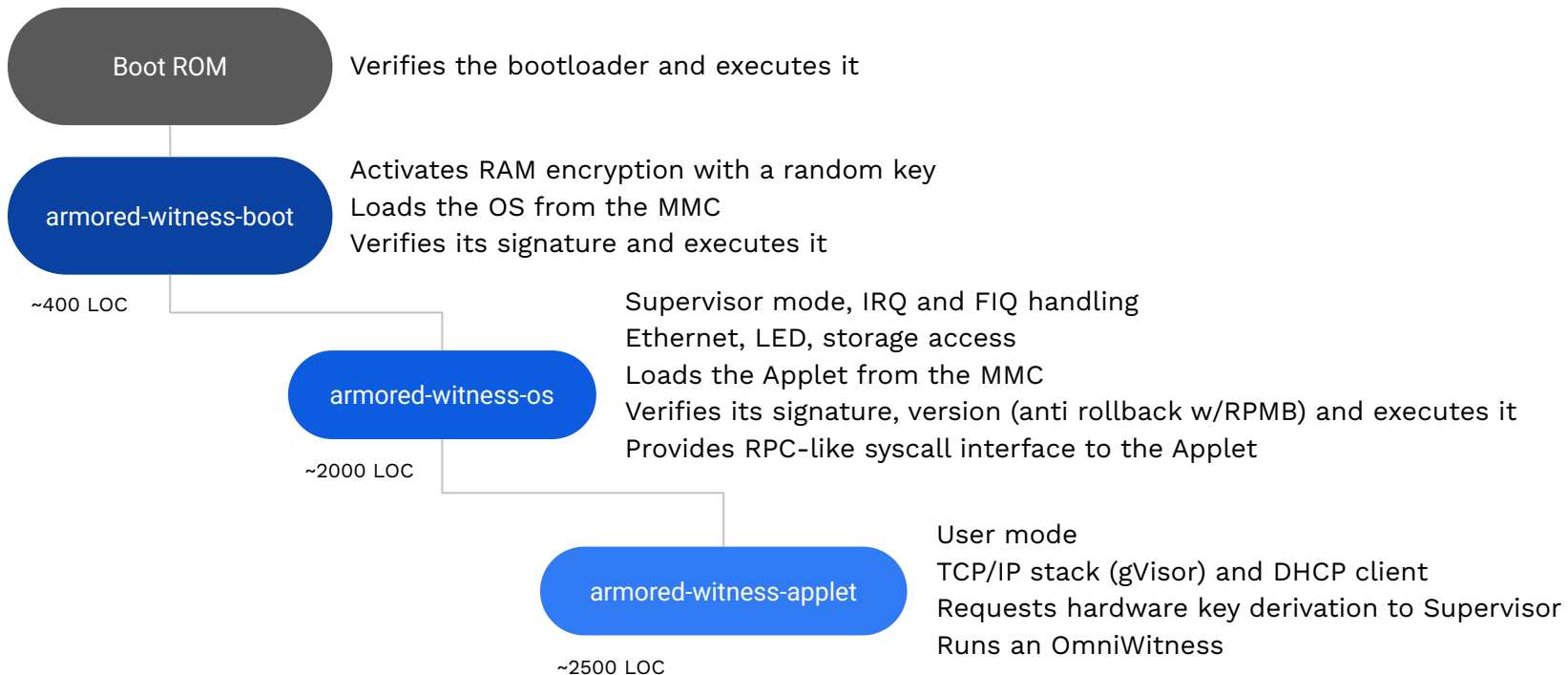
The benefit of this system comes through **removing trust from log operators** to behave honestly, and **placing some of that trust in the witnesses**.

Splitting the trust across multiple parties in this way means that **a larger number of parties must collude to hide malfeasance**, and as other witness implementations/networks start to appear, the number of parties required to collude increases correspondingly.

However, **we can minimise the amount of trust required** to be placed in the Armored Witness by having it be **as transparent as possible** too.



Armored Witness architecture

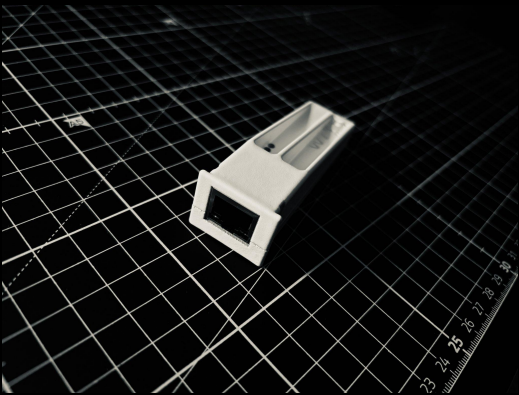


All firmware verification past the Boot ROM stage verify, through FT proof bundles, the presence of one or more trusted signatures (multi party signing) on the release manifest. The release manifest includes the binary hash, log checkpoint, the index of the manifest in the log and its corresponding inclusion proof.

The OS and Applet must be published on the log to be usable on the device.

<https://github.com/transparency-dev/armored-witness>

```
boot: tamago/arm • i.MX6UL
boot: starting kernel@1007dee8
os: tamago/arm • TEE security monitor (Secure World system/monitor)
os: loading applet from MMC storage
os: SM applet verification pub:RWQiFth4tAgsVQT5caaZ6JGgUzZFnwCdeVHe5XpobGWc9XzCJmjJ56t0
os: SM applet loaded addr:0x20000000 entry:0x20081700 size:15323136
os: SM applet started mode:USR sp:0x30000000 pc:0x20081700 ns:false
ta: tamago/arm • TEE user applet
ta: SM starting network
ta: SNVS - Deriving hardware key
ta: Opening storage - CardInfo: {BlockSize:512 Blocks:8388608}
os: SM registering applet event handler g:0x21803900 p:0x21826000
ta: MAC:26:76:04:d4:4d:db IP:10.0.0.1/24 GW:10.0.0.0/24 DNS:8.8.8.8:53
ta: Starting witness...
ta: TA starting ssh server (SHA256:IH0WyXV66dvixxOPDhtAaIAvWg+sQC...) at 10.0.0.1:22
ta: Feeder "go.sum database tree" goroutine started
ta: Feeder "Armory Drive Prod 2" goroutine started
ta: Feeder "rekor.sigstore.dev - 3904496407287907110" goroutine started
ta: Feeder "rekor.sigstore.dev - 2605736670972794746" goroutine started
ta: Feeder "developers.google.com/android/binary_transparency/0" goroutine started
ta: "sum.golang.org: go.sum database tree" grew - @0: - @19895786: 3c9b8f49f56cb...
ta: "armory-drive-log: Armory Drive Prod 2" grew - @0: - @2: 02a14ca4a7313d868a4...
ta: "rekor.sigstore.dev: 2605736670972794746" grew - @0: - @36541297: 0d491271b9...
ta: "pixel_transparency_log: ../binary_transparency/0" grew - @0: - @235: f98458...
ta: "rekor.sigstore.dev: 3904496407287907110" grew - @0: - @4163431: 4d006aa46ef...
ta: "lvfs: lvfs" grew - @0: - @12749: 6ac429c550b8b28b7c65b6e61c99c9c76303d14012...
ta: No checkpoint
```



```
$ ssh 10.0.0.1
```

TA tamago/arm (go1.22.1) • TEE user applet (User Mode)

```
date          (time in RFC339 format)?          # show/change runtime date and time
dns           <fqdn>                            # resolve domain (requires routing)
exit, quit    # close session
hab           <hex SRK hash>                    # secure boot activation (*irreversible*)
help          # this help
led           (white|blue|yellow|green) (on|off) # LED control
mmc           <hex offset> <size>              # MMC card read
reboot        # reset device
stack         # stack trace of current goroutine
stackall      # stack trace of all goroutines
status        # status information
```

```
> status
----- Trusted Applet -----
Runtime .....: go1.22.1 tamago/arm
----- Trusted OS -----
Serial number .....: 3bee6cda358f0c33
Secure Boot .....: false
Revision .....: 70ffeda
Build .....: lcars@lambda on 2024-03-21 08:50:01
Version .....: 1696495801 (2024-03-21 08:50:01 +0000 UTC)
Runtime .....: go1.22.1 tamago/arm
Link .....: true
Witness/Identity .....: DEV:ArmoredWitness-still-tree+e1f17ea8+AZvDSL1C0...
Witness/IP .....: 10.0.0.1
```

```
$ ssh armory
tamago/arm (go1.22.1) • 4112c8d lcars@lambda on 2024-03-08 12:06:34 • i.MX6UL 528 MHz

ntp                <host>                                # change runtime date and time via NTP
tailscale          <auth key> (verbose)?                 # start network servers on Tailscale tailnet
witness            # start/inspect transparency.dev omniwitness
wormhole           (send <path>|recv <code>)             # transfer file through magic wormhole

> ntp time.google.com
2024-03-08T12:06:58Z

> witness
starting omniwitness on :8080 (tamago-example-ephemeral-witness+599e290c+Afszq5oPlHBvi/cyJkGGGHS+r96hhedJK4C71BdqP1G)

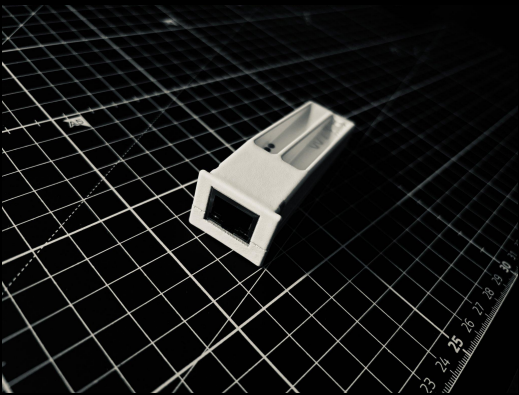
I0308 12:07:04.123431 Feeder "lvfs" goroutine started
I0308 12:07:04.135195 Feeder "go.sum database tree" goroutine started
I0308 12:07:04.142646 Feeder "Armory Drive Prod 2" goroutine started
I0308 12:07:04.148453 Feeder "rekor.sigstore.dev - 3904496407287907110" goroutine started
I0308 12:07:04.157244 Feeder "developers.google.com/android/binary_transparency/0" goroutine started

> witness
Armory Drive Prod 2 2 AqFMpKcxPYaKTmihSfbQvb758iSzJvvJBX5thVJ7r/k=
- armory-drive-log F1Qbj/vNC0bZUS8GUCMAw4A3G0MRU+ZkhVTPm6XTuFST75f2v92/02ilueukoSbFT1zQmCNDT6wor5VB/X5z91bMAA=
- tamago-example-ephemeral-witness 6SKM5Gj/6mUAAAAALhy5ei/8YVJBYTzXWSP8p2ST+868EDQkXUTz40JRSwtz1V0qQ4W6o4g+vneP9rNkiEAN/gQXGXQd9Jz2HQ5JDw==

rekor.sigstore.dev - 3904496407287907110 4163431 TQBqpG78tgfdUdkAsSE3VMUMySUcNAXGw1YdnWovMjk=
- rekor.sigstore.dev wNI9ajBGAiEA8NoGSE0tPoD0tk5kNKQdM4Sxv4L5551vMsbvavFkD1ICIQDW1QsPAS1jGQAqjwOqpWft0m+Iw5P/Kd2ImoUdMgez4g==
- tamago-example-ephemeral-witness 6SKM5Gj/6mUAAAAApV8+I7RoAQSLM0XxxIJ7jZ32mAttz1mLzfiLqogUjniJ59h478pruCKgJbK5sDzQL9JfFeVyG03G4te5bkHDg==

developers.google.com/android/binary_transparency/0 324 vSoSuFDnfUfaKNJne2AtZjQD1CPORv+BLqnSXJE4phE=
- pixel_transparency_log csh42zBFaiEAk7GYrVxnXVZW9UDGMk3vdE0wbHBi2EMUZ0XQ0zz7e3MCIDH0puLouv305pyHW132jJd1ldkClzcS/VUVTbMogsE
- tamago-example-ephemeral-witness 6SKM5Gj/6mUAAAAoDGRDGrZ5UQwnanHsDWhUqpzXY8+6RqTvQczrdMtHr1pJP8nE6G6NvKzWJhcnFc0BKCDITSYWbSAqAaA6EHDg==
go.sum database tree 23470203 fA/vQxRCMYyyCzwGMoSaYipBC0tT0NEv4IpQgL8yCr8=
- sum.golang.org Az3gr1+gARGNedLjWIGz62f3X34EBbUfiPdkuoEZiVqNGKWccGOL9GXKzNBUNgK6MAUx4b5/f3ogAfftTCRV05W54QA=
- tamago-example-ephemeral-witness 6SKM5Gn/6mUAAAAADpA/9F7Te0cuJtPoztsjc+ksmieeezmhxzF+y1+8TMgCbMjBEumBuraML92PLu10TNriB6ocf/pPkyWbzsZIAA==

Lvfs 14770 TVcAhxR0NKIKoKwGAsVSouDSiQ1o21A9LT1ZPZYmnSY=
- lvfs eQJRQZh1vVt9r1JRLxYmHtBz9xtL/0tiRdrOnkakyywcJkKLvooAA2PGUsWi0wuWFXdxTgb1l5VijHQX3DdTos2Q4=
- tamago-example-ephemeral-witness 6SKM5Gn/6mUAAAAZ/uaA9ptWcQKdrT3esieSL9XjAQqMKFVo1EomVTKD+gUYXxVQY6GHKOKbuQdeIAGeyB1gGuQF2xJyJih7NvoBA==
```



// log checkpoint root hash
// log signature

A GOOS=tamago experimental branch for execution with lower memory and CPU extension requirements:

`GOSOFT=1` enables soft float for riscv64

`tiny` enables reduction of RAM to ~6MB

```
GOOS=tamago GOARCH=riscv64 GOSOFT=1
GOOSPKG=github.com/usbarmory/tamago
$TAMAGO build -tags erbium_emu,tiny -trimpath
               -ldflags '-T 0x40010000 -R 0x1000' main.go
```

```
/opt/et/bin/erbium_emu
-elf load main \
-reset_pc $RT0 \
-max cycles -1 \
-single_thread
```

```
> info
SoC .....: Erbium (eb680000) @ 200 MHz (rv64cfimsux)
Runtime .....: go1.26.4 tamago/riscv64 thread 0
RAM .....: 0x40000000-0x40600000 (6 MiB)
Text .....: 0x40010000-0x400f57c8 (917 KiB)
Data .....: 0x402452a0-0x40279bc0 (210 KiB)

> kem
mlkem 3d40397dca5c856349c1e2 .. c8583342afde6a14b2 (15.277ms)
```

```
main.go

package main

import (
    "github.com/usbarmory/kotama/cmd"
    "github.com/usbarmory/tamago-example/shell"
    "github.com/usbarmory/tamago/board/aifoundry/erbium_emu"
)

//go:linkname ramSize runtime/goos.RamSize
var ramSize uint = 6 << 20

func init() {
    _, ramEnd := runtime.MemRegion()
    dataStart, _ := runtime.DataRegion()
    memoryLimit := float64(ramEnd-dataStart) * 0.90
    debug.SetMemoryLimit(int64(math.Round(memoryLimit)))
}

func main() {
    console := &shell.Interface{
        Banner:    banner,
        ReadWriter: erbium_emu.UART0,
    }

    console.Start(true)

    log.Printf("exit")
}
```



What about AI?

⇐ LLMs reading your system (input)

Bare metal means **no OS interfering between your code and the hardware**. An agent can explore fundamental architectural and hardware behavior directly, with the full power of Go packages. What's in the code is what runs, so the model reasons about a closed, fully-specified world instead of guessing at OS state.

```
> You are a hypervisor security researcher running bare
metal inside QEMU microvm.
```

```
[kanzashi] starting agentic QEMU audit...
[kanzashi] initializing claude agent (claude-opus-4-6)
```

```
I'll begin by systematically exploring the QEMU microvm I/O space. The microvm
machine type has a minimal set of peripherals. Let me start by probing known
address ranges for the microvm platform.
```

```
[kanzashi] write32 0xfec00000 <= 0x10 (<nil>)
[kanzashi] write32 0xfec00010 <= 0xffffffff
```

```
Segmentation fault (core dumped)
```

⇒ LLMs writing your system (output)

LLMs choke on sprawling dependency trees and hallucinate package names that don't exist. TamaGo **gives them almost nothing to get wrong**: as few as 3 external dependencies on top of Go's stdlib. **The whole system fits in context**, it leans on the best-represented library in the model's training data, and there's almost no third-party surface left to hallucinate about. And the loop closes: LLM-generated code is inherently risky, but a minimal attack surface bounds the blast radius.

```
> Write a stateless transparency log as TamaGo unikernel,
use only Go stdlib, tamago and go-net packages.
```

```
module github.com/usbarmory/llm-gen-example                                go.mod

go 1.26.4

require (
    github.com/usbarmory/tamago v1.26.4
    github.com/usbarmory/go-net v1.0.0
)

require    github.com/soypat/lneto v0.1.1 // indirect
```



TamaGo

Compiler: <https://github.com/usbarmory/tamago-go>
Library: <https://github.com/usbarmory/tamago>
Documentation: <https://github.com/usbarmory/tamago/wiki>
API: <https://pkg.go.dev/github.com/usbarmory/tamago>
Example: <https://github.com/usbarmory/tamago-example>
Tiny Compiler: <https://github.com/usbarmory/kotama>

GoTEE

Repository: <https://github.com/usbarmory/GoTEE>
Documentation: <https://github.com/usbarmory/GoTEE/wiki>
Example: <https://github.com/usbarmory/GoTEE-example>

Armored Witness

Repository: <https://github.com/transparency-dev/armored-witness>
Bootloader: <https://github.com/transparency-dev/armored-witness-boot>
OS: <https://github.com/transparency-dev/armored-witness-os>
Applet: <https://github.com/transparency-dev/armored-witness-applet>

go-boot

Repository: <https://github.com/usbarmory/go-boot>
HCL: <https://github.com/usbarmory/go-boot/wiki>

TamaGo

Bare metal Go

Secure unikernels:
From Embedded Systems to Confidential VMs

Andrea Barisani

<https://andrea.bio>

andrea@inversepath.com

andrea.barisani@reversesec.com